



GenOS 엔지니어 개발 가이드

개정이력

버전	작성일	개정 내용
v1.0	2026-07-21	초안 작성

목차

- 1. 개요
- 2. GenOS 개발환경
- 3. 공통 개발표준
- 4. MCP 개발 가이드
- 5. 워크플로우 Python 단계 개발 가이드
- 6. 코드 서빙 개발 가이드
- 7. Flowise Custom JS Function 개발 가이드
- 8. 전처리기 개발 가이드
- 9. 평가지표 개발 가이드
- 10. GenOS 서비스 연계 및 재사용
- 11. 배포 및 확인
- 부록 A. 영역별 실행 함수 요약
- 부록 B. 시스템 제공 영역과 사용자 작성 영역
- 부록 C. 영역별 개발 확인 항목

1. 개요

1.1 목적

본 가이드는 GenOS에서 사용자 코드를 작성하고 배포할 때 필요한 개발 규칙과 영역별 작성 방법을 설명한다. 여러 개발자가 같은 실행 기준과 작성 방식을 적용하여 일관된 코드를 작성하고 쉽게 유지보수할 수 있도록 하는 것을 목적으로 한다.

1.2 적용 범위

본 가이드는 GenOS에서 엔지니어가 직접 코드를 작성하는 MCP 도구·MCP 패키지, 워크플로우 Python 단계, 코드 서빙, Flowise Custom JS Function, 전처리기 및 평가지표 영역에 적용한다.

1.3 대상 독자

GenOS에서 Python, JavaScript/TypeScript, Java, Go 또는 C# 기반 사용자 코드를 작성하고 패키징·배포·검증하는 개발자를 대상으로 한다.

1.4 적용 대상 개발 영역

- MCP 도구 (INTERNAL_PYTHON)
- 워크플로우 Python 단계
- 코드 서빙
- MCP 패키지 (CONFIG_JSON): JavaScript/TypeScript 또는 Python
- Spring AI MCP
- Flowise Custom JS Function
- 전처리기
- 평가지표

1.5 규칙 표기 기준

표기	의미
필수	GenOS에서 코드를 실행하거나 배포하기 위해 반드시 적용해야 하는 사항
보안 필수	GenOS가 실행 단계에서 강제하지 않더라도 중요정보 노출과 보안 사고를 방지하기 위해 반드시 적용해야 하는 사항
권장	일관성, 안정성 또는 유지보수성을 위해 적용하는 사항
금지	실행 오류, 보안 문제 또는 운영 장애를 유발할 수 있어 사용하지 않는 사항

2. GenOS 개발환경

2.1 개발 영역별 실행 구조

본 가이드는 GenOS에서 엔지니어가 직접 코드를 작성하고 배포하는 8가지 영역을 다룬다. GenOS는 컨테이너, 호출 연결, 실행 기록 및 인증 기능을 제공한다. 엔지니어는 각 영역에서 정한 함수 또는 모듈 형식에 따라 업무 코드를 작성한다.

#	영역	언어	엔지니어 작성 범위	시스템 제공 범위
1	MCP 도구 (INTERNAL_PYTHON)	Python	시스템이 제공하는 mcp 객체에 @mcp.tool() 함수 여러 개 등록	FastMCP 인스턴스와 ASGI 앱, Streamable HTTP 전송, K8s Pod, PyPI 패키지 설치, 환경변수 주입, Tool 입력 형식 자동 생성
2	워크플로우 Python 단계	Python	def 또는 async def run(data) 하나, 필요 시 sync/async generator	data 입력·다음 단계 전달, OpenTelemetry tracer/span, genos_state 컨텍스트, 함수 유형별 실행 분기, 오류 로깅
3	코드 서빙	Python·Java·JavaScript·Go·C#	Git 저장소 기반 웹 애플리케이션과 빌드·실행 커맨드	언어별 기본 이미지, 저장소 가져오기, 빌드·실행, Gateway 연결, 환경 변수 주입
4	MCP 패키지 (CONFIG_JSON)	JavaScript/TypeScript·Python	npm 패키지의 실행 파일 또는 Python 모듈·스크립트를 stdio MCP 서버로 실행	supergateway stdio↔HTTP 브릿지, K8s Pod, 실행 명령 수행, 도구 자동 동기화
5	MCP Spring AI	Java	@Tool + @ToolParam 메서드 + MethodToolCallbackProvider Bean, 실행 가능한 JAR 업로드	JAR NFS 복사, Spring Boot HTTP 서버 직접 기동, SERVER_PORT 주입, K8s Pod, Tool 입력 형식 생성
6	Flowise Custom JS Function	JavaScript	워크플로우 노드 안에 JavaScript 코드 블록과 명시적 return 작성	vm2 NodeVM 샌드박스, \$input/\$vars/\$flow 주입, async IIFE 래핑, 이전 노드 출력 연결
7	전처리	Python	DocumentProcessor 클래스와 비동기 __call__ 메서드	업로드 파일 경로와 전처리 파라미터 전달, 전처리 결과의 임베딩 연계
8	평가지표	Python	비동기 evaluate(row, predict) 함수	데이터셋 행 순회, 선택한 LLM 또는 워크플로우의 predict 함수 전달, 점수 수집

1은 GenOS가 만든 FastMCP 앱에 사용자 도구 코드를 결합한다. 2는 Python Step 실행 환경이 함수 종류에 따라 실행한다. 3은 선택한 언어의 기본 이미지에서 사용자 Git 저장소를 빌드하고 실행한다. 4는 supergateway가 stdio 방식의 MCP 패키지를 HTTP로 연결한다. 5는 Spring Boot MCP HTTP 서버를 직접 실행한다. 6은 Flowise 컨테이너의 샌드박스에서 실행하며, 7과 8은 각각 전처리 및 평가 실행 환경에서 실행한다.

2.1.1 화면 경로와 표시명

메뉴 경로와 화면 이름은 다음 표기를 사용한다. 괄호 안의 값은 화면에 노출되지 않는 내부 식별자이며 코드 또는 설정 구조를 설명할 때만 사용한다.

개발 영역	화면 경로·표시명	내부 식별자 또는 코드 단위
Python MCP	도구 > MCP 도구, MCP 서버 타입 MCP 도구	INTERNAL_PYTHON
패키지형 MCP	도구 > MCP 서버, MCP 서버 타입 MCP 패키지	CONFIG_JSON
워크플로우 Python	에이전트 > 워크플로우 > Python 단계 추가	Python Step, run(data)
코드 서빙	서빙 > 코드 서빙	Code Serving
Flowise JavaScript	Flowise의 Utilities > Custom JS Function	CustomFunction
전처리기	문서 전처리 > 전처리기	DocumentProcessor
평가지표	평가 > 평가지표 > LLM 또는 평가 > 평가지표 > 워크플로우	evaluate(row, predict)
Python 패키지	관리 > 리소스 > PyPI 패키지	.whl
JavaScript/TypeScript 패키지	관리 > 리소스 > NPM 패키지	npm 패키지
Java 패키지	관리 > 리소스 > JAR 패키지	.jar
A2A 에이전트	에이전트 > A2A 에이전트	Agent Card
프롬프트	도구 > 프롬프트 라이브러리	Prompt 리소스

2.2 지원 언어 및 실행 환경

개발 영역	언어·실행 환경	실행 위치
MCP 도구 (INTERNAL_PYTHON)	Python	GenOS MCP 컨테이너
워크플로우 Python 단계	Python	GenOS 워크플로우 컨테이너
코드 서빙	Python (FastAPI / Flask / Django), Java (Spring Boot), JavaScript (Node / Express / Nest / Fastify), Go (net/http / Gin / Echo / Fiber), C# (ASP.NET Core)	언어별 코드 서빙 컨테이너
MCP 패키지 (CONFIG_JSON)	JavaScript/TypeScript 또는 Python	npm·Python 실행 환경 및 supergateway
MCP Spring AI	Java 21	Spring Boot JAR 직접 실행
Flowise Custom JS Function	JavaScript	Flowise NodeVM 샌드박스
전처리기	Python	GenOS 전처리 실행 환경
평가지표	Python	GenOS 평가 실행 환경

2.3 사용자 작성 영역과 시스템 제공 영역

사용자는 각 영역에서 정해진 함수, Tool 또는 코드 블록을 작성한다. 컨테이너 실행, 호출 연결, 환경변수 설정 및 실행 경로 구성은 GenOS가 제공한다. 세부 구분은 부록 B를 따른다.

2.4 패키지 및 의존성 관리 방식

개발 영역	관리 방식
MCP 도구 (INTERNAL_PYTHON)	관리 > 리소스 > PyPI 패키지에 .whl을 등록하고 MCP 도구에서 선택
워크플로우 Python 단계	워크플로우 컨테이너에 제공된 패키지 사용
코드 서빙	Git 저장소의 언어별 의존성 파일과 빌드 (Build) 커맨드로 관리
MCP 패키지 (CONFIG_JSON)	npm 패키지는 사내 Verdaccio에 게시하고, Python 모듈은 실행 환경에 설치된 패키지 또는 스크립트 코드를 사용
MCP Spring AI	실행 가능한 Spring Boot fat JAR로 빌드
Flowise Custom JS Function	Flowise 외부 모듈 허용 목록에 등록된 패키지 사용
전처리기	전처리 실행 환경에 등록된 Python 패키지 사용
평가지표	평가 실행 환경에서 제공하는 모듈과 predict 함수 사용

2.5 환경변수 및 시크릿 주입 방식

개발 영역	주입 및 접근 방식
MCP 도구 (INTERNAL_PYTHON)	도구 > MCP 도구의 중요 정보 > 환경 변수에 등록하고 <code>os.getenv()</code> 로 조회
워크플로우 Python 단계	리비전 정보 > 환경 변수에 등록하고 선택 언어의 환경 변수 API로 조회
코드 서빙	코드 서빙 리비전 상세 > 환경 설정 > 환경 변수에 등록하고 선택 언어의 환경 변수 API로 조회
MCP 패키지 (CONFIG_JSON)	서버 설정의 env에는 비밀이 아닌 실행 설정만 등록한다. 인증키·비밀번호는 운영 환경에서 정한 보호된 환경 변수 주입 방법을 사용한다.
MCP Spring AI	서버 설정의 env에는 비밀이 아닌 실행 설정만 등록하고 <code>application.yml</code> 에서 참조한다. 인증키·비밀번호는 운영 환경에서 정한 보호된 환경변수 주입 방법을 사용한다.
Flowise Custom JS Function	Flowise <code>\$vars</code> 로 주입
전처리기	전처리기 생성·수정 화면의 환경 변수에 등록하고 <code>os.getenv()</code> 로 조회
평가지표	평가지표 코드에 시크릿을 두지 않고 선택한 LLM 또는 워크플로우를 <code>predict</code> 로 호출

2.6 개발 영역 선택 기준

개발 목적	권장 영역
LLM 에이전트가 호출할 도구 (검색·계산·외부 API) — Python 익숙	1. MCP 도구 (INTERNAL_PYTHON)
같은 도구지만 npm 생태계·기존 JS 라이브러리 활용	4. JS/TS MCP
같은 도구지만 사내 Spring Boot 자산·기존 Java 라이브러리 활용	5. Spring AI MCP
워크플로우 안에서 여러 단계로 분기·분석(Python 위주)	2. 워크플로우 Python 단계
워크플로우 안에서 짧은 JavaScript 코드만 사용(이전 노드 결과 처리, 분기, 항목 변환)	6. Flowise Custom JS Function
외부 시스템에서 직접 호출할 자체 HTTP API (Python·Java·JavaScript·Go·C#)	3. 코드 서빙
LLM, Tool 및 Skill을 조합한 독립 에이전트 HTTP 서비스	3. 코드 서빙
업로드 문서를 검색·임베딩에 사용할 chunk 목록으로 변환	7. 전처리기
LLM 또는 워크플로우 예측 결과를 데이터셋 행별 숫자 점수로 평가	8. 평가지표

3. 공통 개발표준

3.1 명명 규칙

대상	규칙	예시
Python 함수·변수	snake_case	load_document, chunk_size
Python 클래스	PascalCase	DocumentProcessor
Python 상수	UPPER_SNAKE_CASE	MAX_RETRY_COUNT
JavaScript/TypeScript 함수·변수	lowerCamelCase	loadDocument, chunkSize
JavaScript/TypeScript 클래스·타입	UpperCamelCase	RequestValidator
JavaScript/TypeScript 전역 상수	CONSTANT_CASE	DEFAULT_TIMEOUT_MS
Java 클래스	UpperCamelCase	WeatherService
Java 메서드·변수	lowerCamelCase	getWeather, serviceId
Java 상수	UPPER_SNAKE_CASE	DEFAULT_TIMEOUT_MS
환경변수	{SYSTEM}_{PURPOSE}_{UNIT} 형식의 UPPER_SNAKE_CASE	JIRA_API_TOKEN, CRM_REQUEST_TIMEOUT_MS

MCP 도구의 외부 식별자는 구현 언어와 관계없이 다음 형식을 사용한다.

```
{domain}_{action}_{resource}[_{qualifier}]
```

토큰	의미	예시
domain	업무 또는 외부 시스템 경계	jira, crm, billing, customer
action	도구가 수행하는 단일 동작	get, list, search, create, update, delete, validate
resource	처리 대상	issue, customer, invoice, account
qualifier	같은 동작을 추가로 구분해야 할 때만 사용하는 한정자	by_id, summary, status

```
# 사용
jira_search_issue
crm_get_customer_by_id
billing_create_invoice

# 사용하지 않음
getData                # 도메인과 대상이 드러나지 않음
jira_issue              # 동작이 없음
search_jira_issue       # 토큰 순서가 다름
crm_get_customer_info_data # 의미가 중복됨
```

GenOS는 MCP 함수명을 도구 이름으로 사용하며 같은 함수명이 있으면 등록하지 않는다. 도구 이름·설명·입력 형식은 tools/list에 표시되므로 배포 후 이름을 변경할 때는 사용하는 곳에 미치는 영향을 확인한다. 위 이름 형식은 GenOS가 자동으로 검사하지 않으며, 여러 개발자가 같은 방식으로 이름을 작성하기 위한 기준이다.

환경변수는 사용 시스템과 목적을 이름만으로 확인할 수 있게 작성한다. TOKEN, URL, KEY1처럼 사용처를 알 수 없는 이름은 사용하지 않는다.

3.2 실행 함수 작성 규칙

개발 영역	실행 함수 또는 등록 방식
MCP 도구 (INTERNAL_PYTHON)	시스템 제공 mcp 객체에 @mcp.tool()이 적용된 def 또는 async def 함수
워크플로우 Python 단계	모듈 최상단의 run(data) 함수
코드 서빙	Git 저장소의 웹 애플리케이션과 빌드·실행 커맨드
MCP 패키지 (CONFIG_JSON)	server.tool(..., async handler) 등록
MCP Spring AI	@Tool 메서드와 MethodToolCallbackProvider Bean
Flowise Custom JS Function	노드 내부 JavaScript 코드와 명시적 return
전처리기	인자 없이 생성 가능한 DocumentProcessor와 비동기 __call__(request, file_path, **kwargs)
평가지표	비동기 evaluate(row, predict) 함수

3.3 타입과 입출력 데이터 작성 규칙

- 입력과 반환값은 JSON으로 바꿀 수 있는 값으로 작성한다.
- Python은 타입힌트를 작성하고, JavaScript/TypeScript는 Zod로 입력 항목을 정의한다.
- dataclass, Pydantic 객체, numpy 배열 또는 언어별 객체 인스턴스를 직접 반환하지 않는다.

MCP Tool, 코드 서빙 공개 API, 워크플로우의 초기 입력, 전처리기의 파라미터 및 평가지표의 데이터셋 column은 다음 항목을 표로 작성한다. 내부에서만 사용하는 보조 함수에는 이 표를 작성하지 않아도 된다.

항목	작성 내용
필드명	실제 요청 또는 입력 형식의 key
출처	MCP 인자, HTTP body, Workflow data, dataset column 등
타입	str, int, list[str] 등
필수 여부	필수 또는 선택
null 허용	허용 또는 불허
값 정리	공백 제거, 대소문자 변환 등
제약	길이, 범위, 허용값
실패 처리	예외, HTTP 400/422, 오류 결과 등

필드명	출처	타입	필수	null	값 정리	허용 범위	실패 처리
customer_id	MCP 인자	str	O	X	양끝 공백 제거	1~50자	ValueError
limit	MCP 인자	int	X	X	없음	1~100, 기본값 20	ValueError
status	MCP 인자	str	X	X	소문자 변환	active, inactive	ValueError

사용자 또는 외부 시스템에서 전달되는 값은 사용하는 위치에서 자료형과 허용 범위를 확인한다. 문자열을 실행하거나 쿼리에 결합하지 않는다.

```
request_id = str(data.get("request_id", "")).strip()
if not request_id or len(request_id) > 100:
    raise ValueError("request_id must be 1 to 100 characters")

# 값은 SQL 문자열에 연결하지 않고 parameter로 전달한다.
row = await session.execute(
    text("SELECT status FROM service_request WHERE request_id = :request_id"),
    {"request_id": request_id},
)
```

3.4 주석 및 함수 설명 작성 규칙

- Python MCP 도구는 함수 요약, Args: 및 Returns:가 포함된 docstring을 작성한다.
- JavaScript/TypeScript MCP는 도구 설명과 각 Zod 필드 설명을 작성한다.
- Spring AI MCP는 @Tool(description=)과 @ToolParam(description=)을 작성한다.
- GenOS가 직접 호출하는 run, DocumentProcessor.__call__, evaluate와 외부에 공개하는 함수에는 입력, 반환값 및 실제로 발생시키는 오류를 설명한다.
- JavaScript/TypeScript는 다른 모듈에서 사용하는 함수나 타입의 사용 방법을 이름과 타입만으로 알기 어려울 때 JSDoc을 작성한다. 모듈 내부의 단순 함수에는 JSDoc을 작성하지 않아도 된다.
- Java는 외부에 공개하는 도구와 service API에 Javadoc을 작성한다. @Tool과 @ToolParam으로 같은 내용이 충분히 설명된 경우에는 중복 문장을 추가하지 않는다.
- 주석은 코드가 수행하는 동작을 반복하지 않고, 해당 구현을 선택한 이유나 호출자가 지켜야 하는 제약을 기록한다.

함수와 도구 설명의 첫 문장은 다음 형식으로 작성한다.

```
{대상}을 {식별 기준 또는 조건}으로 {동작}하고 {결과}를 반환한다.
```

```

@mcp.tool()
async def jira_get_issue(issue_key: str) -> dict:
    """Jira 이슈를 이슈 키로 조회하고 현재 상태와 담당자를 반환한다.

    Args:
        issue_key: Jira 이슈 키. `PROJECT-123` 형식을 사용한다.

    Returns:
        `issue_key`, `summary`, `status`, `assignee`를 포함한 dict.

    Raises:
        ValueError: issue_key 형식이 올바르지 않은 경우.
        TimeoutError: Jira API가 설정된 시간 안에 응답하지 않은 경우.
    """
    ...

```

데이터를 처리한다, API를 호출한다, 사용자 정보를 가져온다처럼 대상·조건·결과를 알 수 없는 설명은 사용하지 않는다. MCP의 description과 입력 형식은 도구를 호출할 때 함께 제공된다. 다른 실행 함수의 docstring에는 입력 key, 반환 필수 key, 단위·범위 및 실행 시 주의사항을 적는다.

```

async def evaluate(row: dict, predict: callable) -> float:
    """데이터셋의 기준 키워드 포함 비율을 0~1 점수로 반환한다.

    Args:
        row: `question`, `expected_keywords` column을 포함한 데이터셋 행.
        predict: 질문을 받아 LLM 또는 Workflow 결과를 반환하는 비동기 함수.

    Returns:
        float: 포함된 기준 키워드 수를 전체 기준 키워드 수로 나눈 값.

    Raises:
        ValueError: 필수 column이 없거나 기준 키워드가 비어 있는 경우.
    """
    ...

```

3.5 비동기 처리 규칙

- 외부 입출력이 있는 Python 함수는 `async def`로 작성한다.
- JavaScript/TypeScript MCP handler는 Promise를 반환하는 비동기 함수로 작성한다.
- 동기 라이브러리를 사용해야 하는 경우 별도 thread로 실행하여 이벤트 루프 차단을 방지한다.

3.6 외부 시스템 호출 규칙

- 모든 외부 호출에는 제한 시간(timeout)을 설정하고, 호출 코드 또는 연계 문서에 다음 시간을 기록한다.

항목	작성 내용
호출자 전체 제한	현재 요청 또는 Step이 사용할 수 있는 전체 시간
connect timeout	연결 수립 제한
read 또는 total timeout	응답 수신 제한
재시도 대상	timeout, HTTP 502/503/504 등
재시도 제외	입력 오류, 인증 실패, 일반 HTTP 4xx
최대 시도 횟수	최초 호출을 포함한 전체 시도 횟수

```

timeout = httpx.Timeout(connect=3.0, read=20.0, write=5.0, pool=3.0)

async with httpx.AsyncClient(timeout=timeout) as client:
    response = await client.get(url)

```

예를 들어 전체 처리 시간이 30초이면 개별 호출은 10초로 제한하고 최초 호출을 포함해 최대 두 번만 시도할 수 있다. timeout과 HTTP 502-503-504만 다시 시도하며, HTTP 400-401-403-404-422와 입력 오류는 다시 시도하지 않는다. 다시 시도하는 간격은 첫 실패 후 500ms처럼 전체 처리 시간 안에서 정한다. 이 수치는 작성 예시이며 모든 연계에 같은 값을 적용하지 않는다. - GenOS 내부 Workflow, MCP 및 Serving 호출은 Gateway 경로와 Bearer 토큰을 사용한다. - Kubernetes Service DNS를 직접 호출하지 않는다. 직접 호출하면 GenOS의 인증과 실행 기록이 빠진다. - 사용자 입력으로 호출 URL을 구성해야 하는 경우 https scheme과 사전에 정한 host만 허용한다.

```

from urllib.parse import urlparse

ALLOWED_HOSTS = {"api.company.example"}
parsed = urlparse(target_url)
if parsed.scheme != "https" or parsed.hostname not in ALLOWED_HOSTS:
    raise ValueError("target_url is not allowed")

```

3.7 환경변수 및 중요정보 관리 규칙

- API Key, 토큰, 비밀번호 및 접속정보를 코드·패키지·저장소에 하드코딩하지 않는다.
- 중요정보는 각 개발 영역의 환경변수 또는 변수 주입 기능을 사용한다.
- 로그와 오류 응답에 중요정보가 포함되지 않도록 한다.
- 환경변수 누락 시 실제 값을 포함하지 않은 메시지로 즉시 실패시킨다.

```

import os

api_token = os.environ["JIRA_API_TOKEN"]

```

3.8 로깅 규칙

- Python은 GenOS 로거를 사용하고 print()를 사용하지 않는다.
- stdio 기반 JavaScript/TypeScript MCP는 stdout을 JSON-RPC 전용으로 유지하고 로그는 stderr에 기록한다.
- Java는 SLF4J를 사용하고 System.out.println()을 사용하지 않는다.

애플리케이션 로그에는 다음 운영 필드만 기록한다.

```

event, trace_id, request_id, resource_id, status,
duration_ms, item_count, upstream_status, error_code, error_type

```

다음 값은 로그에 기록하지 않는다.

Authorization, Cookie, access_token, refresh_token, api_key, password,
전체 request/response body, 사용자 질문과 LLM 응답 전문, 문서 원문,
개인정보, DB 오류 원문, 인증정보가 포함된 query string

```
logger.warning(  
    "CRM request timed out",  
    extra={  
        "event": "upstream_timeout",  
        "trace_id": trace_id,  
        "resource_id": "crm",  
        "duration_ms": 10_000,  
        "error_type": "TimeoutException",  
    },  
)
```

코드 서빙의 기본 로깅 경로는 request body, response body 및 query string을 기록할 수 있으므로 사용자 애플리케이션이 동일 원문을 추가로 기록하지 않는다. 배포 시 적용된 로깅 middleware와 중요정보 마스킹 설정을 확인하며, GenOS가 사용자 로그의 중요정보를 자동으로 마스킹한다고 가정하지 않는다.

3.9 오류 처리 규칙

사용자 개발 코드의 오류 코드는 {영역 코드}-{공통 오류 코드} 형식으로 작성한다. 영역 코드는 오류가 발생한 개발 위치를 나타내고, 공통 오류 코드는 실패 원인을 나타낸다.

이 형식은 사용자 코드의 반환값과 로그에 사용하는 식별자다. GenOS 관리 API가 반환하는 8자리 오류 코드를 바꾸지 않으며, {영역 코드}-{공통 오류 코드}를 GenOS의 사용자 메시지 파일에서 자동 변환해 준다고 가정하지 않는다.

3.9.1 영역 코드

영역 코드	개발 영역	적용 대상
01	MCP	INTERNAL_PYTHON, JavaScript/TypeScript 및 Spring AI MCP Tool
02	워크플로우	워크플로우 Python 단계
03	코드 서빙	코드 서빙에서 구현한 HTTP API 및 스트리밍 처리
04	Flowise	Flowise Custom JS Function
05	전처리기	DocumentProcessor 구현 코드
06	평가지표	평가지표 구현 코드

MCP Tool에서 외부 API 통신 오류가 발생하면 01-00020001, 워크플로우 Python 단계에서 같은 오류가 발생하면 02-00020001을 사용한다. 언어가 달라도 같은 개발 영역에는 같은 영역 코드를 사용한다.

3.9.2 공통 오류 코드

GenOS의 API 호출 코드에서 사용하는 통신 오류 중 사용자 개발 코드에도 그대로 적용할 수 있는 다음 코드만 사용한다. 독자가 다른 GenOS 오류 코드를 찾아 임의로 추가하지 않는다.

공통 오류 코드	사용 조건	예
00020001	외부 API와 통신하지 못했거나 정상적인 HTTP 응답을 받지 못함	연결 실패, DNS 오류, timeout, HTTP 비정상 응답
00020002	외부 API와 통신은 완료했으나 응답이 실행 실패를 나타냄	HTTP 응답은 정상이지만 응답 본문의 code가 실패를 나타냄
00020003	통신 오류가 아닌 나머지 오류	입력 처리, JSON 변환, 업무 처리, 파일 처리 및 예상하지 못한 내부 오류

입력 오류, 인증 오류, 리소스 없음과 같은 항목에 별도 숫자 코드를 만들지 않는다. 모두 해당 영역의 00020003으로 처리하고 msg에 사용자가 확인할 내용을 작성한다. 예를 들어 MCP Tool의 LLM 출력 JSON 변환 실패는 01-00020003, Workflow의 필수 입력 누락은 02-00020003으로 기록한다.

외부 API 호출 결과는 다음 순서로 구분한다.

1. 연결 실패, timeout 또는 HTTP 비정상 응답이면 00020001을 사용한다.
2. HTTP 통신은 정상적으로 끝났지만 응답 본문이 실행 실패를 나타내면 00020002를 사용한다.
3. 응답 JSON 변환 실패나 사용자 코드 처리 실패는 통신 자체의 실패가 아니므로 00020003을 사용한다.

3.9.3 오류 종류와 처리 방법

오류 종류는 사용자 안내 문구와 HTTP 상태를 정하는 데 사용한다. 통신 오류가 아닌 항목은 종류가 달라도 공통 오류 코드 00020003을 사용한다.

오류 종류	사용 기준	공통 오류 코드	다 시도	코드 서빙 HTTP 상태	사용자 안내 예
INPUT	필수값 누락, 형식 또는 허용 범위 오류	00020003	X	400 또는 422	입력한 고객번호 형식을 확인해 주세요.
AUTH	인증 실패 또는 접근 권한 없음	00020003	X	401 또는 403	해당 기능을 사용할 권한이 없습니다.
RESOURCE	요청한 업무 대상을 찾을 수 없음	00020003	X	404	요청한 고객 정보를 찾을 수 없습니다.
STATE	현재 상태에서는 요청한 변경을 수행할 수 없음	00020003	X	409	현재 상태에서는 이 작업을 처리할 수 없습니다.
UPSTREAM_COMMUNICATION_RETRY	외부 API 연결 실패, timeout 또는 HTTP 502·503·504 응답	00020001	O	502, 503 또는 504	외부 서비스 응답이 지연되고 있습니다. 잠시 후 다시 시도해 주세요.
UPSTREAM_COMMUNICATION_NO_RETRY	외부 API의 HTTP 400·401·403·404·422 응답	00020001	X	연계 API 기준에 따라 변환	외부 서비스 요청을 처리하지 못했습니다. 입력값 또는 권한을 확인해 주세요.
UPSTREAM_EXECUTION	외부 API 통신은 완료했으나 응답 본문이 실행 실패를 나타냄	00020002	조건부	502	외부 서비스가 요청을 처리하지 못했습니다. 잠시 후 다시 시도해 주세요.
LIMIT	호출 횟수 또는 처리량 제한 초과	00020003	조건부	429	요청이 많아 처리하지 못했습니다. 잠시 후 다시 시도해 주세요.
INTERNAL	분류하지 못한 코드 오류 또는 내부 처리 실패	00020003	X	500	요청을 처리하지 못했습니다. 잠시 후 다시 시도해 주세요.

실제 `error_code`에는 영역 코드를 붙인다. 예를 들어 코드 서빙의 입력 오류는 `03-00020003`, MCP Tool의 외부 API 실행 실패는 `01-00020002`로 작성한다.

3.9.4 오류 응답 형식

오류 객체를 반환할 수 있는 영역에서는 다음 항목을 사용한다.

```
{
  "error_code": "01-00020001",
  "msg": "외부 서비스 응답이 지연되고 있습니다. 잠시 후 다시 시도해 주세요.",
  "detail": "weather_api timeout"
}
```

항목	작성 기준
error_code	3.9.1절의 영역 코드와 3.9.2절의 공통 오류 코드를 조합함
msg	사용자가 이해할 수 있는 문장으로 작성하며 가능한 경우 항상 반환함
detail	원인 확인에 필요한 값만 정리하여 선택적으로 반환함

detail에 예외 원문을 그대로 넣지 않는다. 사용자 입력, LLM 출력, 문서 원문, SQL, 토큰, 내부 URL, stack trace 및 외부 시스템 응답 본문을 제외한다. 이러한 값이 포함될 가능성이 있으면 detail을 응답에서 빼고 내부 로그에 정리한 정보만 기록한다.

3.9.5 영역별 전달 방법

개발 영역	오류 전달 방법	유의사항
MCP	isError: true와 함께 content[].text에 오류 객체를 JSON 문자열로 반환함	LLM이 결과를 읽을 수 있으므로 msg와 안전한 detail만 포함함
Workflow	일반 함수는 data["error"]에 오류 객체를 넣어 반환함	다음 Step이 data.get("error")로 분기할 수 있음
Workflow 스트리밍	마지막 단계에서 오류로 스트리밍을 끝낼 때 {"event": "error", "data": {...}}를 전달할 수 있음	다음 Step으로 오류 값을 넘겨야 하면 error 이벤트 대신 마지막 result.data.error에 넣음
코드 서빙	HTTP 상태와 함께 응답 본문의 error_code, msg, 필요한 경우 detail을 반환함	GenOS 채팅 연계에서는 msg만 최종 오류 이벤트로 전달될 수 있으므로 내부 로그에도 같은 코드를 기록함
Flowise Custom JS Function	{ "error": { ... } } 객체를 반환함	뒤의 Node가 error 값을 읽도록 흐름을 구성함
전처리기	같은 오류 코드를 내부 로그에 기록한 뒤 예외를 발생시킴	반환값은 list[dict]여야 하므로 오류 객체를 반환하지 않음
평가지표	같은 오류 코드를 내부 로그에 기록한 뒤 예외를 발생시킴	반환값은 숫자 점수여야 하므로 오류 객체나 오류 문자열을 반환하지 않음

워크플로우의 마지막 Python 단계에서 스트리밍 오류로 종료할 때는 다음 형식을 사용할 수 있다.

```
yield {
  "event": "error",
  "data": {
    "error_code": "02-00020001",
    "msg": "외부 서비스 응답이 지연되고 있습니다. 잠시 후 다시 시도해 주세요.",
  },
}
return
```

error 이벤트는 스트림으로 전달되지만 다음 Step의 data를 바꾸지 않는다. 다음 Step에서 오류를 처리해야 하는 Workflow는 {"event": "result", "data": {**data, "error": {...}}}를 마지막에 전달한다.

3.9.6 미분류 오류 처리 예

다음 예는 Workflow에서 LLM Tool 출력의 JSON 변환에 실패한 경우다. 통신 오류가 아니므로 Workflow 영역 코드 02와 미분류 코드 00020003을 조합한다.

```
ERROR_CODE = "02-00020003"
ERROR_TYPE = "LLM_TOOL_OUTPUT_INVALID_JSON"

try:
    tool_result = json.loads(raw_tool_result)
except json.JSONDecodeError as exc:
    logger.warning(
        "Tool output JSON parsing failed",
        extra={
            "error_code": ERROR_CODE,
            "error_type": ERROR_TYPE,
            "error_detail": f"line={exc.lineno}, column={exc.colno}",
            "trace_id": trace_id,
        },
    )
    return {
        **data,
        "error": {
            "error_code": ERROR_CODE,
            "msg": "도구 실행 결과를 처리하지 못했습니다. 잠시 후 다시 시도해 주세요.",
            "retryable": False,
            "trace_id": trace_id,
        },
    }
```

원본 `raw_tool_result`와 `str(exc)`는 반환값이나 사용자 응답에 포함하지 않는다. 운영 확인을 위해 원문이 반드시 필요하면 접근 권한과 보관 기간이 제한된 별도 저장소를 사용한다.

3.9.7 내부 로그와 사용자 응답 분리

구분	포함하는 항목	포함하지 않는 항목
내부 로그	<code>error_code</code> , <code>error_type</code> , 정리한 <code>error_detail</code> , <code>trace_id</code> , <code>resource_id</code> , <code>upstream_status</code> , <code>duration_ms</code>	토큰, 비밀번호, 개인정보, 전체 요청-응답 본문
Tool 또는 API 오류 응답	<code>error_code</code> , <code>msg</code> , <code>retryable</code> , 필요한 경우 안전하게 정리한 <code>detail</code> 과 <code>trace_id</code>	<code>error_type</code> , 예외 원문, <code>stack trace</code> , 내부 URL, DB 오류 원문
최종 사용자 화면	사용자가 이해할 수 있는 안내 문구, 필요한 경우 문의용 <code>trace_id</code>	오류 원문, 내부 예외 이름, 패키지명, 테이블명, 서버 주소

내부 로그와 Tool 또는 API 응답에는 같은 {영역 코드}-{공통 오류 코드}를 기록한다. 최종 사용자 화면에는 오류 코드가 필요한 경우에만 문의용 정보로 표시하고, 기본적으로는 사용자 안내 문구를 표시한다.

MCP는 SDK가 지원하는 경우 `isError: true`로 실패임을 표시한다. `content`에는 오류 코드와 사용자 안내 문구를 JSON으로 반환한다. Workflow는 다음 Step에서 분기해야 할 때 `data["error"]`에 같은 구조를 넣는다. 예상하지 못한 예외를 {} 또는 None으로 바꾸지 않는다.

3.10 공통 금지 사항

- **stdout에 일반 로그 출력** — Python의 `print()`, JS/TS MCP의 `console.log`는 로그가 수집되지 않거나 MCP JSON-RPC 응답을 손상시킬 수 있다. Python은 `Logger.getLogger(__name__)`, JS/TS MCP는 `console.error(stderr)`, Java는

SLF4J LoggerFactory를 사용한다. Flowise에서는 `console.log`와 `console.error`를 사용할 수 있다.

- **전역 가변 상태** — 모든 pod와 컨테이너는 동시 요청을 처리한다. 함수·모듈 밖의 mutable 전역은 race condition의 원인이 된다. Flowise Custom JS Function은 실행마다 새 NodeVM이 생성되므로 `globalThis`를 지속 캐시로 사용하지 않는다.
- **무한 루프 / 무한 retry** — 외부 호출에는 반드시 timeout을 설정한다. 재시도 횟수와 exponential backoff도 함께 명시한다.
- **GenOS 내부 서비스 직접 호출** — Workflow, MCP, Serving 및 다른 Workflow는 모두 **Gateway 경로**(`/api/gateway/...`)와 **Bearer 토큰**으로 호출한다. K8s Service DNS를 직접 호출하면 인증과 실행 기록을 거치지 않으므로 사용하지 않는다.
- **민감정보 로그 기록** — 토큰·비밀번호·세션 값·문서 원문·개인정보를 로그에 기록하지 않는다. 입력 전체 대신 요청 식별자, 처리 건수, 상태와 같이 운영에 필요한 값만 기록한다.

4. MCP 개발 가이드

4.1 MCP 개발 방식

GenOS의 MCP 서버 생성 화면에서는 서버 타입을 MCP 도구, MCP 패키지 또는 외부 서버 연결로 선택한다. 이 장에서는 코드를 작성하는 MCP 도구와 MCP 패키지를 다룬다. MCP 도구의 내부 식별자는 `INTERNAL_PYTHON`이고, JavaScript/TypeScript와 Spring AI 패키지를 실행하는 MCP 패키지의 내부 식별자는 `CONFIG_JSON`이다.

4.1.1 MCP 도구 (INTERNAL_PYTHON) 실행 방식

도구 > MCP 도구의 코드에 Python 도구 함수를 등록하고 GenOS가 제공하는 FastMCP 서버에서 실행한다.

4.1.2 MCP 패키지 (CONFIG_JSON) 실행 방식

MCP 서버 생성 화면에서 서버 타입 MCP 패키지를 선택하고 실행 템플릿을 NPX 패키지, Python 모듈, Spring AI (Java) 또는 직접 입력으로 정한다. NPX 패키지는 npm 실행 파일을, Python 모듈은 `python3 -m <module_name>` 형식의 모듈을 실행한다. NPX와 Python 템플릿에서는 필요한 경우 스크립트 코드를 함께 등록할 수 있다.

4.1.3 Spring AI 실행 방식

Spring Boot + Spring AI 기반으로 MCP 서버를 작성해 **JAR 빌드** → 관리 > 리소스 > JAR 패키지에 **업로드** → 도구 > MCP 서버 > MCP 서버 생성에서 **서버 타입 MCP 패키지, 실행 템플릿 Spring AI (Java) 선택** 순서로 등록한다. GenOS는 JAR를 NFS에서 복사한 뒤 Java 실행 환경에서 Spring Boot HTTP 서버를 직접 실행하고 `SERVER_PORT`를 넣어 준다. `spring-boot-starter-actuator`를 포함해 `/actuator/health`를 제공한다.

4.2 MCP 도구 (INTERNAL_PYTHON) 구현

4.2.1 기본 구현

도구 > MCP 도구에서 다음 형식의 Python 도구를 등록한다. 이후 도구 > MCP 서버 > MCP 서버 생성에서 서버 타입을 MCP 도구로 선택하고 사용할 MCP 도구를 지정한다. MCP 도구의 내부 식별자는 `INTERNAL_PYTHON`이다.

```
@mcp.tool()
async def get_service_status(service_id: str) -> dict:
    """서비스 식별자에 대한 상태 조회 결과를 반환한다.

    Args:
        service_id (str): 조회할 서비스의 식별자.

    Returns:
        dict: 서비스 식별자와 현재 상태.
    """
    return {"service_id": service_id, "status": "available"}
```

GenOS는 컨테이너 시작 시 FastMCP 인스턴스 mcp와 ASGI 앱을 먼저 생성한 후 사용자 코드를 같은 모듈에 결합한다. 따라서 사용자 코드에는 FastMCP import, mcp = FastMCP(...) 및 별도 ASGI app 생성을 작성하지 않는다. 새 인스턴스를 만들면 시스템 app과 도구가 서로 다른 MCP 인스턴스에 연결될 수 있다.

저장 후 리비전 배포를 수행한다. 배포 상태가 Online으로 전환되면 워크플로우 노드에서 해당 도구를 사용할 수 있다.

4.2.2 PyPI 패키지 등록

필요한 Python 패키지는 관리 > 리소스 > PyPI 패키지 메뉴에서 .whl 파일을 먼저 등록하고 MCP 도구에서 선택한다. 사용자 코드에서 pip install을 실행하지 않는다.

```
# 잘못된 예 - 코드 안에서 설치 시도
import subprocess
subprocess.run(["pip", "install", "requests"]) # 잘못된 예 무시되거나 권한 오류

# 올바른 예 - 사전 등록된 패키지 import
import httpx # ✓ 리소스에서 httpx 선택했다면
```

주요 패키지	비고
httpx	비동기 HTTP 호출 — requests 보다 우선
beautifulsoup4	HTML 파싱 (예: example-mcp-open-url)
pydantic	입력 검증 (v2 권장)
pandas / numpy	데이터 처리 — 결과는 반드시 dict.to_dict() / tolist() 로 변환 후 반환

작성 규칙: Package 추가 시 pyproject.toml 또는 requirements.txt를 함께 갱신한다. Package 목록이 불일치하면 폐쇄망 배포에서 오류가 발생할 수 있다.

4.3 JavaScript/TypeScript MCP 패키지 구현

4.3.1 기본 구현

```
npm init -y
npm install @modelcontextprotocol/sdk zod
mkdir -p src && touch src/index.ts
```

```
// src/index.ts
import { McpServer } from "@modelcontextprotocol/sdk/server/mcp.js";
import { StdioServerTransport } from "@modelcontextprotocol/sdk/server/stdio.js";
import { z } from "zod";

const server = new McpServer({
  name: "request-validation-mcp",
  version: "0.1.0",
});

server.tool(
  "validate_request",
  "요청 식별자의 형식을 확인한다.",
  { requestId: z.string().min(1).describe("확인할 요청 식별자") },
  async ({ requestId }) => ({
    content: [{
      type: "text",
      text: JSON.stringify({ requestId, valid: true }),
    }],
  }),
);

const transport = new StdioServerTransport();
await server.connect(transport);
```

```
// package.json (주요 필드)
{
  "name": "genos-mcp-request-validation",
  "version": "0.1.0",
  "bin": { "genos-mcp-request-validation": "dist/index.js" },
  "files": ["dist"],
  "type": "module",
  "scripts": { "build": "tsc", "start": "node dist/index.js" }
}
```

패키지를 빌드하고 게시한 후 도구 > MCP 서버 > MCP 서버 생성에서 서버 타입 MCP 패키지, 실행 템플릿 NPX 패키지를 선택한다. 서버 설정에 아래 JSON을 입력한다. MCP 패키지의 내부 식별자는 CONFIG_JSON이다.

```
{
  "command": "npx",
  "args": ["-y", "genos-mcp-request-validation"],
  "env": {
    "LOG_LEVEL": "INFO"
  }
}
```

서버 설정의 env 값은 설정 JSON에 포함되므로 비밀이 아닌 실행 설정만 작성한다. 인증키·비밀번호 등 시크릿은 운영 환경에서 정한 보호된 환경변수 주입 방법을 사용하며, 보호된 주입 방법이 정해지지 않은 경우 env에 입력하지 않는다.

4.3.2 패키지 배포

GenOS 사내 npm 레지스트리는 Verdaccio(11mops-npm-registry-service:4873)를 사용한다. 사외 엔지니어는 공개 npm 또는 별도 사내 레지스트리에 패키지를 게시한 후 GenOS 관리자 화면에서 등록한다.

```
# 사내 Verdaccio 사용 시
npm login --registry http://<verdaccio-host>:4873
npm publish --registry http://<verdaccio-host>:4873
```

작성 규칙:

- **실행 파일 정보 필수** — package.json의 "bin"이 없으면 npx 실행 시 *could not determine executable to run* 오류가 발생한다.
- **모듈 방식 일치** — ESM 문법을 사용하는 패키지는 "type": "module"을 지정한다. CommonJS를 사용하는 패키지는 이를 생략할 수 있으며, 한 패키지에서 두 방식을 혼용하지 않는다.
- **배포 파일 지정** — TypeScript 빌드 결과인 dist/를 files에 포함하고 bin이 실제 게시 파일을 가리키게 한다. src/는 실행에 필요하지 않으면 게시하지 않는다.
- **실행 명령 확인** — command를 비워 둘 수 없다. 표준 실행 예시는 npm 패키지의 npx, Python MCP의 python, Spring AI의 java다. 선택한 기본 이미지에 없는 실행 파일은 지정하지 않는다.
- **scoped package 사용 가능** — @scope/name도 사용할 수 있다. scoped 여부와 관계없이 게시된 package의 package.json에 실행 가능한 bin과 배포 대상 파일이 포함돼 있어야 한다.

4.3.3 Python 모듈 및 스크립트 코드 실행

Python으로 작성한 stdio MCP 서버는 도구 > MCP 서버 > MCP 서버 생성에서 서버 타입 MCP 패키지, 실행 템플릿 Python 모듈을 선택한다. 설치된 Python 패키지의 모듈을 실행할 때는 다음과 같이 설정한다.

```
{
  "command": "python3",
  "args": ["-m", "company_mcp.server"],
  "env": {
    "LOG_LEVEL": "INFO"
  }
}
```

별도 패키지 없이 단일 Python 파일을 실행할 때는 스크립트 코드에 stdio MCP 서버 코드를 작성하고, 서버 설정의 실행 파일명이 화면의 스크립트 파일명과 일치하도록 설정한다. 스크립트 코드는 선택 사항이며, 설치된 모듈만으로 실행할 수 있으면 작성하지 않는다. stdout은 MCP JSON-RPC 통신에 사용하므로 일반 로그는 stderr에 기록한다.

4.4 Spring AI MCP 패키지 구현

4.4.1 기본 구현

```
<!-- pom.xml -->
<dependency>
  <groupId>org.springframework.ai</groupId>
  <artifactId>spring-ai-starter-mcp-server-webmvc</artifactId>
  <version>1.1.3</version>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

```
// src/main/java/com/example/mcp/McpApplication.java
package com.example.mcp;

import org.springframework.ai.tool.method.MethodToolCallbackProvider;
import org.springframework.ai.tool.ToolCallbackProvider;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.Bean;

@SpringBootApplication
public class McpApplication {
    public static void main(String[] args) {
        SpringApplication.run(McpApplication.class, args);
    }

    @Bean
    public ToolCallbackProvider tools(WeatherService weatherService) {
        return MethodToolCallbackProvider.builder()
            .toolObjects(weatherService)
            .build();
    }
}
```

```
// src/main/java/com/example/mcp/WeatherService.java
package com.example.mcp;

import org.springframework.ai.tool.annotation.Tool;
import org.springframework.ai.tool.annotation.ToolParam;
import org.springframework.stereotype.Service;

@Service
public class WeatherService {

    @Tool(name = "weather_get_current", description = "도시명으로 현재 날씨를 조회하고 기온을 반환한다.")
    public String getWeather(
        @ToolParam(description = "도시명 (예: 서울)") String city) {
        // 실제 호출은 RestTemplate 또는 WebClient 로
        return "{\"city\":\"" + city + "\", \"temp\":22}";
    }
}
```

빌드한 JAR를 관리 > 리소스 > JAR 패키지에 업로드한다. 이어서 도구 > MCP 서버 > MCP 서버 생성에서 서버 타입 MCP 패키지, 실행 템플릿 Spring AI (Java)를 선택하고 JAR 패키지 선택에서 업로드한 파일을 지정한다. 서버 설정에는 다음 값을 입력한다.

```
{
  "command": "java",
  "args": ["-jar", "/app/server.jar"],
  "env": {
    "SPRING_PROFILES_ACTIVE": "prod"
  }
}
```

GenOS 의 init.sh 가 선택된 JAR 을 NFS 에서 /app/server.jar 로 복사한다.

4.4.2 JAR 빌드 및 배포

```
mvn clean package -DskipTests
# 산출물: target/<artifact>-<version>.jar
```

JAR 작성 규칙:

- **Spring Boot fat JAR** — spring-boot-maven-plugin으로 `java -jar` 실행이 가능한 단일 JAR를 생성한다.
- **파일명 일치** — JAR 패키지 선택에서 지정한 파일명이 실제 업로드 파일명과 일치해야 한다. GenOS는 해당 파일을 `/app/server.jar`로 복사해 실행한다.
- **Actuator health 제공** — spring-boot-starter-actuator를 포함하고 인증 없이 `/actuator/health`가 정상 응답하도록 한다.

Maven groupId, artifactId, version, pom.properties 및 MANIFEST.MF는 빌드와 사내 패키지 추적에 사용할 수 있지만 GenOS가 이 값의 일치 여부를 검사한다고 가정하지 않는다.

4.5 Tool 입력 형식 및 설명 작성

4.5.1 Tool 설명 공통 기준

GenOS는 MCP 서버의 `tools/list` 응답에서 도구의 name, description, inputSchema를 가져온다. Agent가 MCP Tool을 사용할 때도 이 정보를 함께 전달한다. 따라서 description은 관리자 화면에 표시하는 소개 문구가 아니라 LLM이 사용자 요청에 맞는 도구를 선택할 때 사용하는 정보로 작성한다.

description에는 다음 내용을 순서대로 작성한다.

1. 어떤 업무 대상을 처리하는지 작성한다.
2. 어떤 조건이나 식별값으로 동작하는지 작성한다.
3. 어떤 항목을 반환하거나 어떤 데이터를 변경하는지 작성한다.
4. 이름이 비슷한 Tool이 있으면 이 Tool을 선택해야 하는 경우와 선택하지 않아야 하는 경우를 작성한다.

첫 문장은 다음 형식으로 작성한다. 선택 기준이나 변경 효과를 추가해야 하면 두 번째 또는 세 번째 문장으로 나눈다.

```
{대상}을 {식별 기준 또는 조건}으로 {동작}하여 {결과 항목}을 반환한다.
{이 Tool을 사용하는 경우}. {사용하지 않는 경우 또는 데이터 변경 여부}.
```

```
# 올바른 예
JQL 조건으로 Jira 이슈를 검색하여 이슈 키, 제목, 상태와 담당자 목록을 반환한다.
여러 이슈를 조건으로 조회할 때 사용한다. 특정 이슈 한 건을 조회할 때는 jira_get_issue를 사용한다.
이 Tool은 Jira 데이터를 변경하지 않는다.

# 잘못된 예
Jira 이슈를 처리한다.
Jira API를 호출한다.
이슈 검색 Tool이다.
```

Tool 입력 항목의 description에는 입력값의 뜻만 반복하지 않고 형식, 허용 범위, 단위 및 기본값을 작성한다.

입력 항목	올바른 설명	사용하지 않는 설명
issue_key	Jira 이슈 키. PROJECT-123 형식을 사용한다.	이슈 키
limit	반환할 최대 이슈 수. 1~50이며 기본값은 20이다.	개수
start_date	조회 시작일. YYYY-MM-DD 형식을 사용하며 한국 표준시를 기준으로 한다.	시작 날짜
include_closed	종료된 이슈를 포함하려면 true로 설정한다. 기본값은 false다.	종료 포함 여부

내부 API 경로, 클래스명, 사용 라이브러리처럼 Tool 선택에 필요하지 않은 구현 정보는 description에 작성하지 않는다. 인증정보, 내부 서버 주소 및 개인정보 예시도 포함하지 않는다.

4.5.2 MCP 도구 (INTERNAL_PYTHON)

```
@mcp.tool() # ① 인자 없는 데코레이터
async def fetch_weather(city: str, days: int = 1) -> str: # ② async + 타입힌트
    """도시명으로 N일 날씨 예보를 조회하여 날짜, 기온과 날씨 상태를 반환한다.

    현재 날씨만 조회할 때는 weather_get_current를 사용한다.
    이 Tool은 날씨 데이터를 변경하지 않는다.

    Args:
        city (str): 조회할 도시명. 한글 또는 영문을 사용한다. 예: "서울".
        days (int, optional): 조회할 예보 일수. 1~7이며 기본값은 1이다.

    Returns:
        str: `{"city": "서울", "forecast": [...]}` 형태의 JSON 문자열.
    """
    ...
```

항목	규칙	이유
함수명	{domain}_{action}_{resource}[_{qualifier}] 형식의 snake_case	함수명이 도구 식별자로 공개되므로 도메인·동작·대상을 같은 순서로 표시
파라미터명	snake_case, 동사형 금지	입력 항목의 뜻을 이름으로 알 수 있어야 함
타입힌트	파라미터와 반환값에 모두 작성	누락하면 입력 형식이 만들어지지 않거나 Any로 처리될 수 있 음
반환 타입	str / int / float / bool / dict / list 중 하나	JSON으로 바꿀 수 있어야 하며 dataclass·numpy·pandas 객체는 직접 반환하지 않음
Docstring	4.5.1절의 설명과 Google 스타일 Args:, Returns:, Raises:를 작성	Tool 설명과 입력 항목이 LLM의 도구 선택 및 인자 생성에 사 용됨
async	외부 I/O 도구는 async def 사용	def와 async def가 모두 등록되지만, 동기 네트워크·파일 I/O는 이벤트 루프를 차단할 수 있음

4.5.3 JavaScript/TypeScript

@modelcontextprotocol/sdk의 server.tool 또는 setRequestHandler를 사용할 수 있다. 입력 형식을 자동으로 만들 수 있는 server.tool 사용을 권장한다.

```

server.tool(
    "weather_get_forecast",                // ① snake_case
    "도시명으로 N일 날씨 예보를 조회하여 날짜, 기온과 날씨 상태를 반환한다. 현재 날씨만 조회할 때는 weather_get_current를 사용한다.", // ② description
    {                                       // ③ zod 입력 항목
        city: z.string().describe("조회할 도시명. 한글 또는 영문을 사용한다. 예: 서울."),
        days: z.number().int().min(1).max(7).default(1).describe("조회할 예보 일수. 1~7이며 기본값은 1이다."),
    },
    async ({ city, days }) => {           // ④ async 함수, 비구조화 분해
        const data = await fetchWeather(city, days);
        return {
            content: [{ type: "text", text: JSON.stringify(data) }],
        };
    },
);

```

model에는 해당 서버의 /v1/models 응답에서 확인한 모델 식별자를 사용한다. 코드에 특정 모델명을 고정하지 않고 LLM_MODEL_ID 와 같은 환경변수로 전달한다.

항목	규칙	이유
도구 이름	{domain}_{action}_{resource}_{qualifier} 형식의 snake_case	공개 도구 식별자에서 도메인·동작·대상을 같은 순서로 확인
description	대상·조건·결과를 작성하고, 비슷한 Tool과 구분이 필요하면 선택 기준을 추가	LLM이 사용자 요청과 description을 비교하여 사용할 Tool을 선택함
입력 확인	zod로 입력 항목과 범위를 정의	잘못된 입력을 Tool 실행 전에 확인
반환	{ content: [{ type: "text", text: "..."}] }	MCP spec. JSON 결과는 text 안에 stringify
async	모든 핸들러 async	stdio transport 가 Promise 기다림

4.5.4 Spring AI

```

@Service
public class JiraService {

    @Tool(
        name = "jira_search_issue",
        description = "JQL 조건으로 Jira 이슈를 검색하여 이슈 키, 제목, 상태와 담당자 목록을 반환한다. "
            + "여러 이슈를 조건으로 조회할 때 사용하며 특정 이슈 한 건은 jira_get_issue를 사용한다."
    )
    public String searchIssue(
        @ToolParam(description = "검색에 사용할 JQL 문장. 예: project = PAY AND status = Open")
        String jql,
        @ToolParam(description = "반환할 최대 이슈 수. 1~50이며 기본값은 20이다.", required = false)
        Integer limit
    ) {
        // ...
    }
}

```

항목	규칙	이유
클래스	@Service 또는 @Component	Spring DI 로 ToolCallbackProvider 가 발견
메서드명	Java 코드에서는 {action}{Resource} 형식의 lowerCamelCase	Java 언어 관례를 유지
외부 도구명	@Tool(name=)에 {domain}_{action}_{resource} [_{qualifier}] 지정	다른 MCP 구현과 공개 식별자 형식을 통일
@Tool	4.5.1절에 따라 description 작성	LLM이 사용자 요청과 설명을 비교하여 호출할 Tool을 결정함
@ToolParam	모든 인자에 형식·범위·단위·기본값을 포함한 description 작성	LLM이 입력값을 올바른 형식으로 생성할 수 있도록 함
required = false	선택 인자 명시	빠진 인자에 대한 LLM 추론 가능
반환 타입	Spring AI가 JSON으로 바꿀 수 있는 단순 값 또는 응답 DTO	반환 항목을 고정할 때는 필드가 정의된 DTO를 사용하고, 호출 결과가 문자열이어야 할 때만 JSON 문자열을 사용
예외	도구 안에서 catch, JSON 에러 반환	미처리 예외는 HTTP 500

4.6 환경변수 설정

4.6.1 MCP 도구 (INTERNAL_PYTHON)

```
import os

@mcp.tool()
async def search_jira(query: str) -> str:
    """JIRA 이슈를 검색한다."""
    base_url = os.environ["JIRA_BASE_URL"]          # MCP 도구의 중요 정보 > 환경 변수에서 주입
    token    = os.environ["JIRA_API_TOKEN"]         # 시크릿도 동일 경로
    ...
```

- 시크릿 값은 도구 > MCP 도구에서 해당 도구의 중요 정보 > 환경 변수에 입력한다. 코드에 시크릿 값을 평문으로 입력하는 것을 금지한다.
- 환경변수 이름 앞에 사용 시스템명을 붙인다. 예: JIRA_*, SAMSUNG_CARD_*.
- 필수 환경변수가 설정되지 않은 상태로 프로그램이 동작하도록 작성하지 않는다. os.environ["KEY"]를 사용하거나 명시적으로 예외를 발생시킨다. os.getenv에 기본값을 지정하여 오류를 숨기는 방식을 사용하지 않는다.

4.6.2 JavaScript/TypeScript

```
// 코드에서는 process.env를 통해 실행 설정을 조회한다.
const baseUrl = process.env.CRM_BASE_URL;
if (!baseUrl) throw new Error("CRM_BASE_URL required");
```

```
// MCP 서버 생성 화면의 서버 설정 JSON에서 env로 주입
{
  "command": "npx",
  "args": ["-y", "genos-mcp-request-validation"],
  "env": {
    "CRM_BASE_URL": "https://crm.company.example",
    "LOG_LEVEL": "INFO"
  }
}
```

규칙	이유
서버 설정의 env에는 비밀이 아닌 실행 설정만 입력	설정 JSON에 인증정보가 남는 것을 방지
보호된 방법으로 주입한 시크릿은 process.env로 접근	코드에 하드코딩 금지
PATH 덮어쓰기 금지	supergateway가 사용하는 PATH가 바뀌면 npx를 실행할 수 없음
부팅 시 필수 env 검증	누락 시 즉시 throw 해서 컨테이너가 빨리 실패 (silent default 금지)

4.6.3 Spring AI

```
@Service
public class JiraService {
    @Value("${jira.base-url}") // application.yml 또는 env 에서 주입
    private String baseUrl;

    @Value("${jira.api-token}")
    private String apiToken;
    // ...
}
```

```
# application.yml
server:
  port: ${SERVER_PORT:8080}

jira:
  base-url: ${JIRA_BASE_URL}
  api-token: ${JIRA_API_TOKEN}
```

CONFIG_JSON의 env에는 JIRA_BASE_URL과 같은 비밀이 아닌 실행 설정만 입력한다. JIRA_API_TOKEN과 같은 시크릿은 운영 환경에서 정한 보호된 환경변수 주입 방법으로 전달하고 application.yml에서 참조한다. GenOS는 MCP HTTP port를 SERVER_PORT로 주입하므로 Spring Boot server port 설정에서 이 값을 사용한다.

4.7 오류 처리 및 로깅

MCP Tool의 오류 결과는 호출 프로그램뿐 아니라 LLM에도 전달될 수 있다. LLM이 실패 원문을 사용자 답변에 그대로 포함할 수 있으므로 예외 메시지, 내부 URL 및 외부 시스템 응답을 Tool 결과에 넣지 않는다. 3.9절의 오류 코드와 사용자 안내 문구를 사용하고 상세 원인은 내부 로그에 기록한다.

4.7.1 MCP 도구 (INTERNAL_PYTHON)

```
import json
from common.logger import Logger # GenOS 가 컨테이너에 자동 주입
logger = Logger.getLogger(__name__)

@mcp.tool()
async def fetch_weather(city: str) -> str:
    """도시명으로 현재 날씨를 조회하여 기온과 날씨 상태를 반환한다."""
    try:
        async with httpx.AsyncClient(timeout=10.0) as client:
            r = await client.get(f"https://api.weather.com/{city}")
            r.raise_for_status()
            return r.text
    except httpx.TimeoutException:
        error_code = "01-00020001"
        error_type = "WEATHER_UPSTREAM_TIMEOUT"
        logger.warning(
            "weather API timed out",
            extra={"error_code": error_code, "error_type": error_type},
        )
        return json.dumps({
            "error": {
                "error_code": error_code,
                "msg": "날씨 정보 조회가 지연되고 있습니다. 잠시 후 다시 시도해 주세요.",
                "retryable": True,
            }
        }, ensure_ascii=False)
    except httpx.HTTPStatusError as e:
        is_input_error = 400 <= e.response.status_code < 500
        error_code = "01-00020001"
        error_type = (
            "WEATHER_INPUT_INVALID" if is_input_error else "WEATHER_UPSTREAM_UNAVAILABLE"
        )
        logger.error(
            "weather API returned an error",
            extra={
                "error_code": error_code,
                "error_type": error_type,
                "upstream_status": e.response.status_code,
            },
        )
        return json.dumps({
            "error": {
                "error_code": error_code,
                "msg": (
                    "입력한 도시명을 확인해 주세요."
                    if is_input_error
                    else "날씨 정보를 조회하지 못했습니다. 잠시 후 다시 시도해 주세요."
                ),
                "retryable": not is_input_error,
            }
        }, ensure_ascii=False)
```

규칙	설명
예상한 예외는 도구 안에서 처리한다	입력 오류, 외부 시스템 timeout 등 처리 방법을 정한 오류는 try/except 후 오류 코드와 사용자 안내 문구를 반환한다. 예상하지 못한 예외는 성공 결과로 바꾸지 않는다.
timeout 필수	모든 외부 호출에 timeout을 두고 3.6절의 전체 처리 시간 안에서 값을 정함
오류 응답 형식	{"error":{"error_code":"...","msg":"...","retryable":true}} 형식을 사용한다. 예외 원문은 반환하지 않는다.
logger	응답과 같은 error_code를 extra=에 기록한다. error_type, 외부 시스템 상태 및 trace 정보는 내부 로그에만 기록한다.

4.7.2 JavaScript/TypeScript

stdio transport를 사용하므로 `console.log` 사용을 금지한다. stdout은 MCP JSON-RPC 프레임 전용으로 사용한다. 로그는 `console.error`를 사용하여 stderr로 출력한다.

```
// 잘못된 예 stdout 오염 - JSON-RPC 파싱 실패
console.log("debug info");

// ✓ stderr 로만 출력
console.error("[request-validation-mcp] starting up");

server.tool("fetch", "허용된 HTTPS 주소에서 공개 데이터를 조회하여 텍스트로 반환한다.", { url: z.string() },
async ({ url }) => {
  try {
    const r = await fetch(url, { signal: AbortSignal.timeout(10_000) });
    return { content: [{ type: "text", text: await r.text() }] };
  } catch (e) {
    const errorCode = "01-00020001";
    const errorType = "WEB_UPSTREAM_UNAVAILABLE";
    console.error(JSON.stringify({
      event: "web_fetch_failed",
      error_code: errorCode,
      error_type: errorType,
      exception_type: e instanceof Error ? e.name : "UnknownError",
    }));
    return {
      isError: true,
      content: [{
        type: "text",
        text: JSON.stringify({
          error_code: errorCode,
          msg: "외부 데이터를 조회하지 못했습니다. 잠시 후 다시 시도해 주세요.",
          retryable: true,
        }),
      ]},
    ];
  }
});
```

4.7.3 Spring AI

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

@Service
public class JiraService {
    private static final Logger log = LoggerFactory.getLogger(JiraService.class);

    @Tool(name = "jira_search_issue", description = "JQL 조건으로 Jira 이슈를 검색하여 이슈 키, 제목과 상태를 반환한다.")
    public String searchIssue(@ToolParam(description = "JQL") String jql) {
        try {
            String body = restTemplate.getForObject(baseUrl + "/rest/api/2/search?jql=" + jql,
String.class);
            return body;
        } catch (HttpClientErrorException.BadRequest e) {
            String errorCode = "01-00020001";
            String errorType = "JIRA_INPUT_INVALID";
            log.warn("Jira API rejected JQL. error_code={} error_type={} upstream_status={}",
                errorCode, errorType, e.getStatusCode().value());
            return "{\"error\":{\"error_code\":\"\" + errorCode
                + "\",\"msg\":\"JQL 검색 조건을 확인해 주세요.\",\"
                + \"retryable\":false}}";
        } catch (ResourceAccessException e) {
            String errorCode = "01-00020001";
            String errorType = "JIRA_UPSTREAM_TIMEOUT";
            log.warn("Jira API timed out. error_code={} error_type={}", errorCode, errorType);
            return "{\"error\":{\"error_code\":\"\" + errorCode
                + "\",\"msg\":\"Jira 응답이 지연되고 있습니다. 잠시 후 다시 시도해 주세요.\",\"
                + \"retryable\":true}}";
        }
    }
}
```

규칙	이유
LoggerFactory.getLogger(클래스) 사용	System.out.println 금지
e.printStackTrace() 금지	컨테이너 표준 logger 로만 출력
except Exception: pass 패턴 금지	항상 구체 예외 + 로그
HTTP 호출 timeout 설정	RestTemplate 빌드 시 setConnectTimeout, setReadTimeout 명시

4.8 실행 및 검증

4.8.1 MCP 도구 (INTERNAL_PYTHON)

- 1. MCP 서버 배포 후:** 도구 > MCP 서버에서 해당 서버 상세 화면을 열고 테스트를 선택하여 MCP 서버 테스트 화면을 연다. Tool이 보이지 않으면 선택한 MCP 도구와 함수의 @mcp.tool()을 확인한다.
- 2. 도구 호출:** MCP 서버 테스트에서 Tool과 JSON 입력을 선택하여 응답을 확인한다. 응답이 지연되면 동기 I/O 또는 외부 호출 timeout 누락 여부를 확인한다.
- 3. 컨테이너 로그:** `kubectl logs -n <namespace> <mcp-server-pod>` 명령으로 `Logger.getLogger(__name__)`의 출력을 확인한다.

4. **tools/call JSON-RPC** (curl): `bash curl -sS -X POST "${GENOS_URL}/api/gateway/mcp/<server_id>/mcp" \ -H "Content-Type: application/json" \ -H "Authorization: Bearer ${TOKEN}" \ -d '{"jsonrpc":"2.0","id":1,"method":"tools/call", "params":{"name":"get_service_status","arguments":{"service_id":"billing-api"}}}'`

4.8.2 JavaScript/TypeScript

1. **로컬 stdio 실행**: `bash echo '{"jsonrpc":"2.0","id":1,"method":"tools/list"}' | npx genos-mcp-request-validation`
2. **GenOS 등록 후 동기화**: MCP 서버 상세 화면의 중요 정보 > 도구 동기화를 실행한다. 동기화가 완료되면 MCP 서버가 제공하는 도구 목록을 확인한다.
3. **GenOS 등록 후 호출**: 도구 동기화가 끝나면 테스트를 선택하여 MCP 서버 테스트에서 한 번 이상 Tool을 실행한다. 호출이 지연되면 패키지의 stderr와 supergateway 로그를 함께 확인한다.

4.8.3 Spring AI

1. **로컬 HTTP 실행**: `bash java -jar target/server.jar curl -fsS http://localhost:8080/actuator/health`
2. **MCP 연결 확인**: Spring AI에 설정한 MCP HTTP 경로로 초기 연결 요청과 tools/list를 호출한다. Java 방식은 stdio와 supergateway를 사용하지 않는다.
3. **MCP 서버 등록 후**: MCP 서버 상세 화면의 중요 정보 > 도구 동기화로 @Tool 메서드가 도구 목록에 반영됐는지 확인한다.
4. **JAR 구성 확인**: 사용자 JAR는 `java -jar`로 단독 실행 가능한 fat JAR로 빌드하고, JAR 패키지 선택에서 지정한 파일과 실제 업로드 파일이 일치하는지 확인한다.

4.9 잘못된 작성 사례

4.9.1 MCP 도구 (INTERNAL_PYTHON)

```
# 잘못된 예 1) sync 함수에서 blocking I/O
@mcp.tool()
def slow(): time.sleep(5)          # 등록은 가능하지만 실행 중 다른 요청을 지연시킬 수 있음

# 잘못된 예 2) 타입힌트 없음
@mcp.tool()
async def bad(a, b):              # 입력 타입을 확인할 수 없음
    return a + b

# 잘못된 예 3) docstring 없음 / 파라미터 설명 누락
@mcp.tool()
async def get_service_status(service_id: str) -> dict:
    return {"service_id": service_id} # 반환 의미와 파라미터 설명을 확인할 수 없음

# 잘못된 예 4) 전역 mutable
CACHE = {}                        # 동시 요청 시 race condition

@mcp.tool()
async def memoize(k: str, v: str) -> str:
    CACHE[k] = v
    return CACHE[k]

# 잘못된 예 5) 시크릿 하드코딩
@mcp.tool()
async def call(...):
    token = "xoxb-1234..."      # PR 리뷰 차단 사유 - feedback_no_hardcoded_secrets

# 잘못된 예 6) 객체 직접 반환
@mcp.tool()
async def stats() -> "pandas.DataFrame":
    return df                     # JSON으로 바꿀 수 없어 500 오류 발생
                                  # 대신: return df.to_dict(orient="records")
```

4.9.2 JavaScript/TypeScript

```
// 잘못된 예 1) console.log 사용 - stdout 오염
console.log("got request");

// 잘못된 예 2) zod 입력 정의 없이 raw any
server.tool("bad", "", {}, async (args: any) => { ... });
// → 각 입력값의 뜻과 범위를 알 수 없음

// 잘못된 예 3) sync handler
server.tool("slow", "", { ... }, ({ ... }) => { ... });
// → 반환이 Promise 가 아니면 transport 가 응답 못 받음

// 잘못된 예 4) bin 누락
// package.json 에 "bin" 없으면 npx 실행 불가

// 잘못된 예 5) PATH override
"env": { "PATH": "/custom/path" }
// → supergateway 내부 도구 호출 실패
```

4.9.3 Spring AI

```
// 잘못된 예 1) @Tool description 누락
@Tool
public String getWeather(...) { ... } // LLM 호출 결정 불가

// 잘못된 예 2) @ToolParam description 누락
public String getServiceStatus(String serviceId) { ... }
// → 입력 이름만 표시되어 각 값의 의미를 알 수 없음

// 잘못된 예 3) System.out.println 사용
System.out.println("debug"); // 컨테이너 로그 수집 누락

// 잘못된 예 4) 광범위 catch
try { ... } catch (Exception e) {} // 무음 실패 (silent-failure)

// 잘못된 예 5) Spring AI 1.0 사용
// → MCP starter 가 1.1+ 부터 GA. 이전 버전은 미지원

// 잘못된 예 6) actuator 누락
// → /actuator/health 응답 없음 → readiness probe 실패 → Pod 재시작 루프

// 잘못된 예 7) base image에 없는 실행 파일 지정
"command": "custom-java" // executable을 찾지 못해 기동 실패
// → Java image의 "java" 사용
```

5. 워크플로우 Python 단계 개발 가이드

5.1 기본 구현

에이전트 > 워크플로우에서 대상 워크플로우와 리비전을 연 뒤 Python 단계 추가를 선택하고 다음 형식으로 코드를 작성한다. 화면에서 별도의 Step Type을 선택하지 않는다.

```

async def run(data: dict) -> dict:
    """요청 문자열의 앞뒤 공백을 제거하여 다음 Step으로 전달한다."""
    question = data.get("question", "")
    data["question_normalized"] = question.strip()
    return data

```

모듈 최상단에는 반드시 run 함수를 정의한다. 함수명이 다른 경우 Step 실행 엔진에서 run function not found 오류와 HTTP 500 응답이 발생한다.

5.2 실행 함수 유형

엔진은 inspect를 사용하여 함수 유형을 구분한다. 다음 유형 중 하나를 선택하여 일관되게 적용한다.

```

# ① 동기 - 구성이 단순. 짧고 CPU 위주 작업
def run(data: dict) -> dict:
    data["x"] = compute(data["y"])
    return data

# ② async - 외부 I/O가 있는 경우 적용 권장
async def run(data: dict) -> dict:
    async with httpx.AsyncClient(timeout=10.0) as c:
        r = await c.get(data["url"])
    data["html"] = r.text
    return data

# ③ async generator - 토큰 스트리밍 (LLM 응답을 실시간 전달)
async def run(data: dict):
    async for chunk in llm.stream(data["question"]):
        yield {"event": "token", "data": chunk}
    yield {"event": "result", "data": {**data, "text": accumulated}}

# ④ 동기 generator - 드물지만 가능 (CPU 위주 청크 분할)
def run(data: dict):
    for chunk in chunks_of(data["text"]):
        yield {"event": "token", "data": chunk}
    yield {"event": "result", "data": data}

```

패턴	사용 시점	반환·yield 형식
① 동기	CPU 위주, 외부 호출 없음	return dict
② async	외부 API·DB·LLM 단발 호출	return dict
③ async generator	토큰 스트리밍 (UI 실시간)	yield {"event": "token", "data": ...} 반복 + 마지막에 yield {"event": "result", "data": dict}
④ sync generator	청크 단위 처리	③ 과 동일

③·④의 경우 마지막 event: result의 data가 다음 step으로 전달되는 새 data dict가 된다. event: result를 보내지 않으면 generator 실행 전에 전달받은 기존 data가 다음 step으로 이어진다.

5.3 데이터 전달 및 genos_state 사용

```

async def run(data: dict) -> dict:
    # — 사용자 입력 (Flowise overrideConfig 또는 요청 데이터)
    question = data["question"]

    # — 시스템이 자동 주입하는 GenOS 컨텍스트
    state = data.get("genos_state", {})
    trace_id = state.get("trace_id")          # OpenTelemetry trace ID (UUID)
    session_id = state.get("session_id")      # 대화 세션 ID (멀티턴)
    user_id = state.get("genos_user_id")      # GenOS 사용자 ID
    workflow_id = state.get("genos_resource_id")

    # — 비즈니스 로직
    answer = await call_llm(question, user_id=user_id)

    # — 다음 step 에 넘길 키 추가 (기존 키 보존 권장)
    data["answer"] = answer
    return data

```

genos_state 가 담는 표준 키 (시스템 자동 주입, inject_genos_state):

키	의미
trace_id	분산 추적 UUID. 외부 호출에도 같은 값을 전달하여 요청 흐름을 확인할 때 사용
span_id	현재 span hex16
session_id	같은 대화에 속한 요청을 구분하는 값. 일반 Python Step의 이전 대화를 자동으로 불러오지는 않음
name	현재 실행 리소스의 이름
service	현재 실행 서비스의 이름
genos_user_id	호출자 사용자 ID
genos_resource_id	현재 workflow ID
genos_resource_revision_id	워크플로우 리비전 ID
genos_resource_deploy_id	배포 ID
guardrail_applied	가드레일 적용 여부

5.4 A2A 스트리밍 및 상태 전달

다중 에이전트(A2A) 워크플로우에서 **사용자의 추가 입력이 필요한 상태(input-required)**를 상위로 전파할 때, GenOS는 세 가지 채널을 모두 지원한다.

```
# ① status 이벤트로 (스트리밍 토큰과 병행 가능)
async def run(data: dict):
    async for tok in llm.stream(...):
        yield {"event": "token", "data": tok}
    yield {"event": "a2a_status", "data": "input-required"}
    yield {"event": "result", "data": {**data, "text": accumulated}}

# ② result 의 data 안에 (스트리밍과 병행 가능)
async def run(data: dict):
    ...
    yield {"event": "result",
           "data": {**data, "text": accumulated, "a2a_status": "input-required"}}

# ③ 단순 dict return (비스트리밍 기본 유형)
async def run(data: dict) -> dict:
    return {**data, "text": "추가로 어떤 정보가 필요한가요?",
            "a2a_status": "input-required"}
```

유의사항	처리 방법
Exception 발생 + input-required 동시 시도	Exception이 우선 적용되어 failed로 처리되며 명시한 상태는 적용되지 않음
event: token 전송 중 sourceDocuments 전송	UI token 흐름에 영향을 줄 수 있으므로 buffering 후 마지막에 전송
event: end 를 중간에 yield	end 는 가장 마지막에 1회. 그 전에 pending_post_events 가 모두 emit 되어야 함

5.5 컨테이너 환경

항목	값
Base image	python:3.10.19 위 poetry 환경
Pod	container-services/workflow (워크플로우 리비전당 1개)
사용 가능 모듈	asyncio, httpx, json, datetime, re, opentelemetry.*, GenOS Logger
시크릿 주입	워크플로우의 리비전 정보 > 환경 변수

5.6 오류 처리

```
async def run(data: dict) -> dict:
    try:
        result = await call_external_api(data["query"])
    except httpx.HTTPStatusError as e:
        return {
            **data,
            "error": {
                "error_code": "02-00020001",
                "msg": "검색 서비스를 사용할 수 없습니다. 잠시 후 다시 시도해 주세요.",
                "retryable": e.response.status_code >= 500,
            },
        }
    except asyncio.TimeoutError:
        return {
            **data,
            "error": {
                "error_code": "02-00020001",
                "msg": "검색 서비스 응답이 지연되고 있습니다. 잠시 후 다시 시도해 주세요.",
                "retryable": True,
            },
        }

    data["result"] = result
    return data
```

- 예외를 그대로 발생시키면 워크플로우 전체가 실패한다. 다음 Step에서 오류를 처리하려면 dict의 error key로 전달한다.
- 다음 step(예: 분기용 IF 노드)에서 data.get("error")의 값에 따라 처리 경로를 나눈다.

5.7 잘못된 작성 사례

```
# 잘못된 예 함수 이름이 다름
async def main(data: dict) -> dict: ...      # → "run function not found" 500

# 잘못된 예 return + yield 혼용
async def run(data: dict):
    yield {"event": "token", "data": "hi"}
    return data                                # SyntaxError (Python)

# 잘못된 예 전역 import 의 lazy 평가
db = pymongo.MongoClient(...)                # 모듈 로드 시점 == 컨테이너 부팅 시 1회만
                                              # 컨테이너가 실행되는 동안 유지됨
                                              # 단발 호출 인 줄 알았다면 connection leak

# 잘못된 예 data 객체를 새 dict 로 통째 교체
async def run(data: dict) -> dict:
    return {"answer": "..."}                  # genos_state 등 다른 키 손실
                                              # → return {**data, "answer": "..."}

# 잘못된 예 blocking I/O
async def run(data: dict) -> dict:
    requests.get(...)                          # 이벤트 루프 정지
                                              # → httpx.AsyncClient 또는 asyncio.to_thread

# 잘못된 예 print 디버그
async def run(data: dict) -> dict:
    print(data)                                # 로그 시스템에 안 잡힘
                                              # → logger.info("step input", extra={...})
```

6. 코드 서빙 개발 가이드

6.1 실행 구조와 지원 언어

GenOS 코드 서빙은 Git 저장소를 가져온 뒤 선택한 언어의 기본 이미지에서 사용자 웹 애플리케이션을 빌드하고 실행한다. Python service(config, data)는 기존 코드를 실행하기 위한 호환 방식이며, 모든 코드 서빙에서 반드시 사용해야 하는 함수는 아니다.

언어 선택값	애플리케이션 예	의존성 파일 예
Python (FastAPI / Flask / Django)	FastAPI, Flask, Django	requirements.txt, pyproject.toml
Java (Spring Boot)	Spring Boot	pom.xml, build.gradle
JavaScript (Node / Express / Nest / Fastify)	Node, Express, Nest, Fastify	package.json, lock file
Go (net/http / Gin / Echo / Fiber)	net/http, Gin, Echo, Fiber	go.mod, go.sum
C# (ASP.NET Core)	ASP.NET Core	.csproj, .sln

관리 API에는 다섯 언어가 등록돼 있다. Python 외 언어를 배포하기 전에는 해당 template-code-serving-* 이미지가 설치되어 있는지 운영 환경의 시스템 이미지 목록에서 확인한다.

코드 서빙 리비전 상세 화면의 탭은 기본 정보, 컨테이너 상태, 환경 설정, 컨테이너 서비스로 구성된다. 이 장에서 설명하는 언어·빌드·실행·환경 변수 설정은 환경 설정에 입력하고, 워크플로우·전처리·MCP 연계 경로는 컨테이너 서비스에서 설정한다.

6.2 기본 구현

다음은 Python FastAPI 저장소의 기본 예시다. 다른 언어도 0.0.0.0:\$PORT에서 HTTP 서버를 실행하고 상태 확인 경로를 제공한다.

```
# main.py
import os
from fastapi import FastAPI
from pydantic import BaseModel, Field

app = FastAPI()

class ValidationRequest(BaseModel):
    request_id: str = Field(min_length=1, max_length=100)

@app.get("/health")
async def health() -> dict:
    return {"status": "ok"}

@app.post("/requests/validate")
async def validate_request(body: ValidationRequest) -> dict:
    return {"request_id": body.request_id, "valid": True}

if __name__ == "__main__":
    import uvicorn

    uvicorn.run(app, host="0.0.0.0", port=int(os.environ["PORT"]))
```

Python 애플리케이션은 main.py 또는 src/main.py가 있으면 해당 파일을 먼저 실행한다. 두 파일을 사용하지 않거나 다른 실행 명령이 필요하면 코드 서빙 리비전 상세 > 환경 설정의 시작 (Run) 커맨드를 입력한다. Java, JavaScript, Go 및 C# 애플리케이션은 선택한 언어에 맞는 시작 (Run) 커맨드를 입력한다.

6.3 빌드 및 기동 설정

서빙 > 코드 서빙 > 코드 서빙 생성에서 제목, 상세 설명, 관리 그룹, 저장소 유형과 Git 저장소 정보를 입력한다. 생성 후 리비전 추가에서 도커 이미지, 인스턴스 타입, GPU 할당량, 복제본, 브랜치, 커밋 해시를 입력한다. 리비전을 만든 뒤 코드 서빙 리비전 상세 > 환경 설정에서 언어, 빌드 (Build) 커맨드, 시작 (Run) 커맨드 및 환경 변수를 설정한다.

항목	작성 규칙	예시
Git 버전	같은 코드를 다시 받을 수 있도록 커밋 값 사용	a1b2c3d...
빌드 (Build) 커맨드	필요한 패키지 설치와 빌드 명령 입력	pip install -r requirements.txt
시작 (Run) 커맨드	HTTP 서버가 종료되지 않도록 실행	uvicorn main:app --host 0.0.0.0 --port \$PORT
환경 변수	사용자 애플리케이션에 필요한 값을 변수명과 값으로 등록	CUSTOMER_API_URL

GenOS는 실행할 때 PORT를 8080, OPENAPI_PATH를 /openapi.json 기본값으로 주입한다. 두 값은 화면에서 직접 입력하는 등록 항목이 아니다. 애플리케이션은 PORT를 읽어 서버를 실행하고, 같은 이름을 다른 목적으로 사용하지 않는다.

```
# Python
BUILD_COMMAND=pip install -r requirements.txt
START_COMMAND=uvicorn main:app --host 0.0.0.0 --port $PORT

# Node
BUILD_COMMAND=npm ci && npm run build
START_COMMAND=npm run start

# Spring
BUILD_COMMAND=./mvnw -DskipTests package
START_COMMAND=java -jar target/app.jar --server.port=$PORT
```

사용자 Dockerfile은 코드 서빙의 표준 등록 단위가 아니다. OS 패키지 또는 별도 이미지가 반드시 필요한 경우에는 운영 배포 방식과 기본 이미지 변경 절차를 먼저 확인한다.

6.4 HTTP 애플리케이션 작성 기준

- 서버는 0.0.0.0과 GenOS가 넣어 주는 PORT에 연결한다. localhost에만 연결하면 Gateway와 상태 확인 기능이 접근할 수 없다.
- 업무 API의 경로와 요청·응답 항목은 사용자 애플리케이션에서 정한다. /json과 /multipart는 Python 호환 방식에서만 자동으로 제공된다.
- 내부 배포는 GET /health가 HTTP 200을 반환해야 한다. 코드 서빙의 상태 확인 프로그램이 이 경로를 직접 호출한다.
- CPU 사용이 많거나 실행을 오래 기다리는 작업은 사용하는 프레임워크의 worker 또는 thread 기능으로 실행한다. 비동기 처리 함수 안에서 time.sleep, 동기 HTTP 호출 및 장시간 파일 처리를 직접 실행하지 않는다.

코드 서빙은 HTTP 상태와 오류 객체를 함께 반환한다. 다음 예와 같이 외부 API timeout에는 03-00020001, 그 밖의 처리 실패에는 03-00020003을 사용한다.

```
from fastapi.responses import JSONResponse

try:
    result = await call_external_api(body.request_id)
except httpx.TimeoutException:
    return JSONResponse(
        status_code=504,
        content={
            "error_code": "03-00020001",
            "msg": "외부 서비스 응답이 지연되고 있습니다. 잠시 후 다시 시도해 주세요.",
        },
    )
except ValueError:
    return JSONResponse(
        status_code=400,
        content={
            "error_code": "03-00020003",
            "msg": "입력값을 확인해 주세요.",
        },
    )
```

예외 종류와 정리한 상세 원인은 같은 error_code와 함께 내부 로그에 기록한다. repr(exc)나 stack trace를 HTTP 응답 본문에 넣지 않는다.

6.5 GenOS 연계용 표준 경로

코드 서빙 리비전 상세 화면의 컨테이너 서비스에서 코드 서빙을 워크플로우로 사용, 전처리기로 사용 또는 MCP 서버로 사용하도록 설정할 수 있다. 해당 기능을 사용할 때는 설정된 HTTP 경로와 다음 요청·응답 형식을 구현한다. workflow_chat, preprocessor, mcp_list, mcp_call이라는 Python 함수명은 공통으로 정해진 이름이 아니다.

용도	기본 HTTP 경로	요청·응답 기준
워크플로우로 사용	POST /chat	워크플로우 연계 설정에 등록된 요청·응답 항목
전처리기로 사용	POST /preprocess	file_path, file_content_base64, file_name, file_size, params를 받고 {"code":0,"data":[...]} 반환
MCP 서버로 사용 — 도구 목록	POST /mcp/list	{"code":0,"data":{"tools":[{"name":"...", "description":"...", "inputSchema":{"...}}]}} 반환
MCP 서버로 사용 — 도구 호출	POST /mcp/call	{"name":"...", "arguments":{"...}}를 받고 {"code":0,"data":{"content":[{"type":"text","text":"..."}]}} 반환

연계 기능을 사용하지 않으면 해당 경로를 만들 필요가 없다. 경로를 변경할 경우 등록 화면의 호출 경로와 애플리케이션 경로를 같은 값으로 변경한다.

코드 서빙 전처리기는 호출 컨테이너와 파일 저장 경로를 공유한다고 가정하지 않는다. 요청의 file_content_base64를 디코딩하여 파일을 처리하고, file_name과 file_size는 형식·크기 확인에 사용한다. file_path만 열어 처리하는 구현은 사용하지 않는다.

6.6 스트리밍 응답

```
import asyncio
from fastapi.responses import StreamingResponse

async def token_stream():
    for token in ["처리", " 중", "입니다"]:
        yield f"data: {token}\n\n"
        await asyncio.sleep(0.2)

@app.post("/chat/stream")
async def stream_chat() -> StreamingResponse:
    return StreamingResponse(token_stream(), media_type="text/event-stream")
```

호출 프로그램이 중간 결과를 받을 때만 실시간 응답 경로를 제공한다. Proxy가 응답을 모아서 보내는지와 호출 프로그램이 중간 결과를 읽는지를 함께 확인한다.

6.7 환경변수 및 시크릿

```
import os

db_url = os.environ["CUSTOMER_DB_URL"]
api_key = os.environ["CUSTOMER_API_KEY"]
port = int(os.environ["PORT"])
```

- 환경 변수와 시크릿은 코드 서빙 리비전 상세 > 환경 설정 > 환경 변수에 등록하고 .env를 저장소에 커밋하지 않는다.
- GenOS는 BUILD_COMMAND, START_COMMAND, LANGUAGE, PORT, OPENAPI_PATH 등 실행 설정을 환경변수로 전달한다. 애플리케이션에서 같은 이름을 다른 목적으로 사용하지 않는다.

- 사용자 입력과 원본 예외를 환경변수 또는 로그에 복사하지 않는다.

6.8 호출 및 테스트

```
curl -fsS "${GENOS_URL}/api/gateway/code_serving/<id>/health"

curl -fsS -X POST \
  "${GENOS_URL}/api/gateway/code_serving/<id>/requests/validate" \
  -H "Authorization: Bearer ${TOKEN}" \
  -H "Content-Type: application/json" \
  -d '{"request_id":"REQ-20260721-001"}'
```

배포 확인 시 상태 확인 응답뿐 아니라 실제 업무 경로의 정상 입력, 입력 오류 및 외부 시스템 timeout을 각각 실행한다. 예상 HTTP 상태값과 응답 본문의 필수 key를 API 문서에 함께 적는다.

6.9 잘못된 작성 사례

```
# 잘못된 예 1) localhost에만 연결
uvicorn.run(app, host="127.0.0.1", port=8080)

# 잘못된 예 2) 시스템 PORT 무시
uvicorn.run(app, host="0.0.0.0", port=9000)

# 잘못된 예 3) 비동기 처리 함수에서 오래 기다리는 작업 실행
@app.post("/requests/validate")
async def validate_request(body: dict):
    time.sleep(5)
    return requests.post(UPSTREAM_URL, json=body).json()

# 잘못된 예 4) 상태 확인 경로 없음
# /health를 정의하지 않으면 내부 배포 test와 health check가 실패한다.

# 잘못된 예 5) 사용자 앱에서 호환용 경로를 필수 경로로 가정
CODE_SERVING_URL = f"{base_url}/json"
# 사용자 앱의 실제 route와 등록 설정을 사용한다.
```

6.10 코드 서빙 에이전트 구현

6.10.1 적용 범위

코드 서빙은 Git 저장소의 HTTP 애플리케이션을 실행하므로 LLM 호출, Tool 호출 및 대화 상태 처리를 포함한 에이전트 서비스를 구현할 수 있다. GenOS는 저장소 빌드, 컨테이너 실행, 환경 변수 주입 및 Gateway 연결을 제공한다.

에이전트의 요청 처리, LLM 호출, Tool 등록 및 실행 종료 조건은 코드 서빙 애플리케이션에서 구현한다.

6.10.2 저장소 구성

에이전트별 코드와 공통 코드를 다음과 같이 분리한다.

```

agent-service/
├── app/
│   ├── main.py                # FastAPI 앱과 router 등록
│   ├── agents/
│   │   └── {에이전트명}/
│   │       ├── router.py      # HTTP 요청·응답 처리
│   │       ├── schema.py      # 요청·응답 모델
│   │       ├── service.py     # 에이전트 실행 순서
│   │       ├── settings.py    # 환경변수 설정
│   │       └── tools/         # 해당 에이전트 전용 Tool
│   └── common/
│       ├── llm_client.py      # 공통 LLM 호출
│       └── tool_registry.py   # Tool 이름과 실행 함수 등록
├── requirements.txt
└── README.md

```

다음 기준에 따라 코드를 배치한다.

파일·디렉터리	작성 내용
router.py	요청 모델을 받고 service.py를 호출한 뒤 HTTP 또는 SSE 응답을 반환한다. LLM·Tool 실행 순서를 작성하지 않는다.
schema.py	입력값, 기본값 및 응답 항목을 고정한다.
service.py	LLM 호출, Tool 실행, 반복 종료 및 최종 응답 생성을 담당한다.
tools/	해당 에이전트만 사용하는 Tool을 둔다. Tool 이름, 설명 및 입력 형식을 함께 정의한다.
common/	둘 이상의 에이전트에서 사용하는 LLM 클라이언트와 Tool 실행기를 둔다.

6.10.3 Tool 등록

실제로 실행할 수 있는 함수는 Tool 등록부에 등록한다. 에이전트는 등록부에서 자신이 사용할 Tool 이름을 골라 Tool 집합을 구성한다.

```

TOOL_REGISTRY = {
    "credit_get_limit": credit_get_limit,
    "credit_get_usage": credit_get_usage,
}

async def execute_tool(name: str, arguments: dict):
    tool = TOOL_REGISTRY.get(name)
    if tool is None:
        raise ValueError(f"unregistered tool: {name}")
    return await tool(**arguments)

```

- Tool 이름은 LLM에 제공한 이름과 TOOL_REGISTRY의 key를 같게 작성한다.
- Tool 설명에는 대상, 사용 조건, 반환 결과와 비슷한 Tool과의 선택 기준을 작성한다.
- Tool 입력은 실행 전에 schema로 확인한다.
- 둘 이상의 에이전트가 사용하는 Tool은 common/으로 이동하고 한 곳에서 관리한다.

6.10.4 응답 방식

단일 결과만 필요한 에이전트는 일반 JSON 응답을 사용한다. 중간 토큰 또는 Tool 진행 상태를 화면에 보여 주어야 할 때만 SSE를 사용한다. SSE를 사용하는 경우 각 이벤트를 {"event": "...", "data": ...} 형식으로 작성한다.

```
data: {"event": "token", "data": "조회 결과를 확인했습니다."}
```

```
data: {"event": "tool_result", "data": {"name": "credit_get_limit", "status": "completed"}}
```

```
data: {"event": "result", "data": {"text": "최종 안내 문구"}}
```

오류 이벤트에는 3.9절의 코드 서빙 오류 코드를 사용한다.

```
data: {"event": "error", "data": {"error_code": "03-00020001", "msg": "외부 서비스 응답이 지연되고 있습니다. 잠시 후 다시 시도해 주세요."}}
```

SSE 이벤트 이름은 호출 화면과 먼저 합의한 값만 사용한다. 코드 서빙이 임의로 이벤트를 추가해도 GenOS 화면이 자동으로 처리한다고 가정하지 않는다.

6.10.5 에이전트 실행 보안

- LLM이 만든 Tool 이름과 인자를 그대로 실행하지 않고 Tool 등록부와 입력 schema를 통과시킨다.
- 외부 URL을 입력받는 Tool은 https와 허용된 host만 호출한다.
- shell 또는 Python 코드를 실행하는 Tool은 격리된 실행 환경이 준비된 경우에만 제공한다. 코드 서빙 애플리케이션 프로세스에서 LLM이 만든 코드를 직접 실행하지 않는다.
- 사용자별 토큰과 회사 공용 API Key는 같은 변수나 저장소에 섞지 않는다. 각 자격증명을 사용하는 Tool과 접근 범위를 분리한다.
- LLM 요청과 Tool 결과 전체를 로그에 기록하지 않는다. trace_id, Tool 이름, 처리 시간, 결과 상태와 오류 코드만 기록한다.
- Tool 호출 반복 횟수와 외부 호출 timeout을 설정하고 무한 반복이나 무한 재시도를 허용하지 않는다.

6.11 Skill 개발 가이드

6.11.1 Skill의 역할

Skill은 에이전트가 특정 요청을 처리할 때 필요한 절차 지식을 요청 시점에 제공하는 형식이다. Skill 자체가 API를 호출하거나 코드를 실행하지는 않는다. 실제 동작은 Tool이 수행하고, Skill은 어떤 상황에서 어떤 Tool을 어떤 순서와 조건으로 사용할지를 설명한다.

구분	역할	예
Tool	에이전트가 실제로 실행하는 함수	MCP 또는 API로 제공하는 조회 함수
Skill	Tool을 조합하여 결과를 만드는 절차	조회, 정렬 및 결과 작성에 사용할 Tool과 순서를 정한 실행 안내

하나의 Skill에서 여러 Tool을 순서대로 호출하거나 결과에 따라 다른 Tool을 선택할 수 있다. Skill 하나와 Tool 하나를 일대일로 맞추는 필요는 없다.

이 절에서 설명하는 Skill은 SKILL.md 파일로 작성하는 절차 지식을 뜻한다. 에이전트가 사용하는 Tool 집합과 Skill 파일은 서로 다른 항목으로 관리한다.

6.11.2 Skill 파일 구조

Skill 하나는 하나의 디렉터리와 하나의 SKILL.md로 구성한다. 실행 프로그램, 긴 참고 문서 또는 출력용 리소스가 필요한 경우에만 선택 디렉터리를 추가한다.

```
stock-item-inquiry/  
├─ SKILL.md  
├─ scripts/  
├─ references/  
└─ assets/
```

항목	규칙	예
Skill 디렉터리	kebab-case로 작성한다.	stock-item-inquiry
상세 파일	대소문자를 구분하여 정확히 SKILL.md로 작성한다.	stock-item-inquiry/SKILL.md
Skill 식별자	디렉터리명과 동일한 kebab-case로 작성한다.	name: stock-item-inquiry
실행 프로그램	scripts/에 저장한다.	형식 검사 프로그램
긴 참고 문서	references/에 저장한다.	업무 기준 또는 항목 정의
템플릿·리소스	assets/에 저장한다.	결과 템플릿

Skill 디렉터리 안에 별도의 README.md를 만들지 않는다. 설명은 SKILL.md 또는 references/에 작성한다.

Skill 디렉터리명과 name에는 공백, 밑줄 및 대문자를 사용하지 않는다. claude 또는 anthropic이 포함된 이름도 사용하지 않는다. Claude와 OpenAI용 파일을 따로 만들지 않고 동일한 SKILL.md를 사용한다.

6.11.3 SKILL.md 기본 형식

SKILL.md는 YAML 형식의 앞부분과 Markdown 본문으로 구성한다.

```

---
name: stock-item-inquiry
description: 보유 종목의 현재 평가금액이나 손익을 조회할 때 사용한다. "삼성전자 얼마야?"와 같은 요청에 적용한다. 단순 계좌
잔고 확인에는 사용하지 않는다.
---

# 보유 종목 조회

보유 종목의 평가금액과 손익을 조회하여 안내한다.

## Instructions

1. 사용자 요청에서 종목명을 확인한다.
2. `stock_item_inquiry`를 호출한다.
3. 조회 결과에서 평가금액, 증감금액 및 증감률을 확인한다.

## 예외 처리

- 종목을 찾지 못하면 종목명을 다시 확인하도록 안내한다.

## 사용자 안내 문구

- 조회 성공: "{종목명} 현재 평가금액은 {평가금액}원이며, 매입금액 대비 {증감금액}원 ({증감률}%) {상승/하락}했습니다."

## Examples

- Input: "삼성전자 얼마야?"
- Output: `stock_item_inquiry`를 호출하고 평가금액과 증감률을 추출하여 사용자 안내 문구로 답한다.

```

앞부분은 첫 번째 ---와 두 번째 --- 사이에 작성한다. 두 번째 --- 다음부터는 Skill이 선택된 뒤 에이전트가 읽을 실행 절차다.

6.11.4 앞부분 작성 규칙

항목	필수 여부	작성 규칙
name	필수	Skill 디렉터리명과 동일한 kebab-case로 작성한다.
description	필수	수행 기능, 사용 시점, 사용자 요청 예와 사용하지 않는 경우를 작성한다.
license	선택	외부 배포가 필요한 경우 라이선스를 작성한다.
compatibility	선택	대상 제품, 시스템 패키지 및 네트워크 등 필요한 실행 환경을 작성한다.
allowed-tools	선택	Skill에서 접근할 수 있는 Tool을 작성한다.
metadata	선택	author, version, mcp-server, domain, required_tools 등 추가 정보를 작성한다.

프로젝트에서 추가하는 항목은 최상위에 만들지 않고 metadata 아래에 작성한다. required_tools는 Skill 표준의 필수 또는 권장 항목이 아니다. Tool 정보를 검색이나 분류에 사용할 필요가 있을 때 본문의 Tool 설명과 별도로 metadata.required_tools에 작성한다. Tool 이름에는 실제 API 이름을 사용하며, Skill 이름과 달리 snake_case를 사용할 수 있다.

description은 1,024자 이내로 작성하고 XML 태그와 실행 지시를 넣지 않는다. 구체적인 작성 항목은 6.11.6절을 따른다.

6.11.5 단계별 로딩

Skill은 필요한 정보만 다음 순서로 읽는다.

단계	읽는 시점	읽는 내용
1	항상	name과 description
2	Skill이 선택된 때	SKILL.md 본문
3	본문의 지시가 있는 때	scripts/, references/, assets/의 필요한 파일

항상 제공하는 정보에는 name과 description만 포함한다. Skill 선택 조건은 description에 작성하고, 실행 방법은 본문에 작성한다. 본문은 500줄 이내로 작성하는 것을 권장하며 공식 상한은 5,000단어다.

6.11.6 description 작성 규칙

description에는 다음 내용을 작성한다.

1. Skill이 처리하는 업무 영역과 기능을 작성한다.
2. 어떤 사용자 요청에서 사용해야 하는지 작성한다.
3. 사용자가 말할 수 있는 표현이나 관련 파일 형식을 작성한다.
4. 비슷한 Skill에서 잘못 선택될 수 있으면 사용하지 않는 조건과 대신 사용할 Skill을 작성한다.

Skill이 필요한 요청에서 선택되지 않으면 에이전트에게 해당 Skill을 언제 사용하는지 확인하고, 답변에 나타난 description을 기준으로 빠진 사용 조건을 보완한다.

6.11.7 본문 작성 규칙

Skill 본문은 선택된 뒤 에이전트가 그대로 따르는 실행 안내서다.

다음 순서로 작성한다.

1. 제목과 한두 문장의 개요를 작성한다.
2. Instructions에 핵심 절차를 번호가 있는 단계로 작성한다.
3. 각 단계에서 사용할 Tool의 실제 이름, 호출 순서 및 분기 조건을 작성한다.
4. 실패하거나 경계 조건에 해당할 때의 행동을 예외 처리에 작성한다.
5. 최종 결과의 형식과 사용자 안내 문구를 작성한다.

6. 입력과 출력의 예를 Examples에 작성한다.

문장은 조회한다, 확인한다와 같은 명령형으로 작성한다. 중요한 지시는 본문 위쪽에 배치하고, 반드시 지켜야 할 항목은 Critical 절로 분리할 수 있다. 지시에는 필요한 경우 이유를 함께 작성한다. 예를 들어 중복 주문이나 중복 조회가 발생할 수 있으므로 다시 시도하지 않는다고 설명한다.

한 가지 종목이나 사례에만 맞는 절차를 작성하지 않는다. 사용하지 않는 문장은 제거하고, 긴 내용은 references/로 옮긴 뒤 본문에서 필요한 시점과 파일 경로를 안내한다. 형식 검사처럼 결과가 명확한 검증은 scripts/에 프로그램으로 작성하고, 본문에는 실행 시점과 성공 조건을 작성한다.

6.11.8 에이전트에서 Skill 사용

코드 서빙에서 자체 에이전트를 구현하여 Skill을 사용할 때는 다음 기능을 에이전트 애플리케이션에 구현하여 Skill 파일을 요청 처리에 연결한다.

기능	구현 내용
Skill 목록 구성	Skill 디렉터리를 읽어 name과 description 목록을 만든다.
Skill 선택	사용자 요청과 Skill 설명을 모델에 제공하고 사용할 Skill을 선택하게 한다.
상세 내용 불러오기	선택한 Skill의 SKILL.md 본문과 필요한 참고 파일만 불러온다.
모델 입력에 적용	불러온 내용을 현재 요청의 실행 지침으로 모델에 전달한다.
Tool 연결	Skill 본문에 적힌 Tool을 별도로 등록하고, 모델이 선택한 Tool을 기존 Tool 실행 기능으로 호출한다.

즉, Skill은 실행 코드가 아니라 에이전트가 불러와서 따르는 지침이다. 실제 API 조회나 업무 처리는 별도로 등록된 Tool이 수행한다.

7. Flowise Custom JS Function 개발 가이드

Flowise의 Custom JS Function 노드에서 JavaScript 코드를 실행하는 방식이다. 내부 노드 식별자는 CustomFunction이다. 현재 노드의 직접 입력은 \$input, 워크플로우 변수는 \$vars, 세션과 실행 정보는 \$flow로 제공된다. 워크플로우 Python 단계와 달리 Flowise 컨테이너 안에서 실행되므로 사용할 수 있는 모듈과 전역 객체가 다르다.

Custom JS Function은 실행마다 새 NodeVM을 만들며 sandbox timeout은 10초로 고정돼 있다. 외부 호출 timeout은 sandbox 제한보다 짧게 설정하고, 여러 요청 사이에 유지해야 하는 상태는 NodeVM 전역이 아니라 외부 저장소에 둔다.

7.1 기본 구현

Flowise 워크플로우에서 Utilities > Custom JS Function 노드를 추가하고, Javascript Function 입력란에 다음 형식으로 코드를 작성한다.

```
// 사용 가능한 전역: $input, $vars, $flow, console
const text = $input || $vars.question;
return {
  question: String(text).trim(),
  receivedAt: new Date().toISOString()
};
```

반환값은 객체, 문자열 또는 숫자로 작성할 수 있으며 다음 노드의 입력으로 연결된다.

7.2 입력 및 전역 객체

```
// — 입력 —————
$input      // 현재 노드의 직접 입력 (이전 노드 출력)
$vars       // 워크플로우 전체 변수 ($vars.foo, $vars.bar)
$flow       // { chatflowId, sessionId, chatId, rawOutput, input }
// — 이전 노드 출력을 이름으로 —————
$customFunction_0 // 같은 워크플로우의 다른 CustomFunction 노드 결과
$agentAgentflow_1 // Agentflow 노드 결과

// — 표준 —————
console.log / console.error // 컨테이너 로그로 출력
JSON / Promise / Math / Date // 기본 JS 전역

// — 옵션인 (env 로 화이트리스트) —————
require('axios') // TOOL_FUNCTION_EXTERNAL_DEP 에 등록된 npm 만
require('crypto') // TOOL_FUNCTION_BUILTIN_DEP 에 등록된 Node 내장만
```

\$flow.rawOutput은 현재 실행에서 전달된 원본 출력 문자열이다. \$vars는 이전 노드 출력이 아니라 워크플로우에 등록된 공통 변수이므로, 노드 간 데이터 전달에는 \$input 또는 노드 식별자 변수를 사용한다.

7.3 함수 작성 및 반환 규칙

CustomFunction 코드는 **자동으로** `async function() { ... }()` **IIFE로 감싸진다**. 따라서 최상위에서 `return`과 `await`를 모두 사용할 수 있다.

```
// — 동기 —————
const result = compute($vars.x);
return result;

// — 비동기 + 외부 API 호출 —————
const axios = require('axios');
const r = await axios.post(
  `${$vars.GENOS_URL}/api/gateway/code_serving/${$vars.CODE_ID}/requests/validate`,
  { question: $vars.question },
  {
    headers: { Authorization: `Bearer ${$vars.TOKEN}` },
    timeout: 8_000,
  }
);
return r.data;
```

항목	규칙	이유
함수 선언	불필요 (IIFE 자동 래핑)	return 만 쓰면 됨
await	최상위에서 사용 가능	async IIFE 안
반환 타입	string / number / boolean / object / array	JSON으로 자동 변환
예외	try/catch + 에러 객체 반환 권장	uncaught throw 는 워크플로우 전체 실패
console.log	디버그 용도로 사용 가능	Flowise 컨테이너 로그에 출력됨

7.4 외부 API 및 데이터베이스 호출

axios, mysql2/promise, node-fetch 등은 **사전에 허용 목록에 등록한 후** require로 불러올 수 있다. Flowise를 배포할 때 환경변수 TOOL_FUNCTION_EXTERNAL_DEP에 패키지 이름을 쉼표로 구분하여 등록한다.

```
// 데이터 분석 워크플로우 예
const mysql = require('mysql2/promise');

const conn = await mysql.createConnection({
  host: $vars.host,
  port: Number($vars.port),
  user: $vars.user,
  password: $vars.password,
  database: $vars.database,
});

try {
  const [rows] = await conn.execute(
    'SELECT COLUMN_NAME, DATA_TYPE FROM INFORMATION_SCHEMA.COLUMNS WHERE TABLE_SCHEMA = ? AND TABLE_NAME = ?',
    [$vars.database, $vars.table]
  );
  return { columns: rows };
} catch (e) {
  const errorCode = '04-00020003';
  const errorType = 'ANALYTICS_UPSTREAM_UNAVAILABLE';
  console.error(JSON.stringify({
    event: 'analytics_query_failed',
    error_code: errorCode,
    error_type: errorType,
    exception_type: e instanceof Error ? e.name : 'UnknownError',
  }));
  return {
    error: {
      error_code: errorCode,
      msg: '데이터를 조회하지 못했습니다. 잠시 후 다시 시도해 주세요.',
      retryable: true,
    },
  };
} finally {
  await conn.end();
}
```

규칙	이유
시크릿은 \$vars로만 받음	코드에 값을 직접 작성하지 않고 UI의 변수 입력에서 설정
finally 로 connection close	vm2 샌드박스가 자동 close 보장 안 함 → DB 커넥션 누수 위험
timeout 명시	NodeVM의 10초 제한 전에 호출을 종료하도록 10초보다 짧은 timeout 사용
GenOS 내부 호출은 Gateway 경로 사용	K8s Service DNS를 직접 호출하면 인증과 실행 기록을 거치지 않음

7.5 GenOS 서비스 호출

```
// — 다른 워크플로우 호출 —————
const axios = require('axios');
const r = await axios.post(
  `${$vars.GENOS_URL}/api/gateway/workflow/${$vars.WORKFLOW_ID}/run/v2`,
  { question: $vars.question },
  { headers: { Authorization: `Bearer ${$vars.TOKEN}` }, timeout: 8_000 }
);
return r.data.data;    // { text: "...", ...next_step_data }

// — Code Serving 호출 —————
const r2 = await axios.post(
  `${$vars.GENOS_URL}/api/gateway/code_serving/${$vars.CODE_ID}/requests/validate`,
  { question: $vars.question },
  { headers: { Authorization: `Bearer ${$vars.TOKEN}` }, timeout: 8_000 }
);
return r2.data;
```

7.6 요청 간 상태 및 캐시

Custom JS Function 실행마다 새 NodeVM이 생성되므로 `globalThis`, `module` 변수 또는 객체에 저장한 값의 다음 요청 유지가 보장되지 않는다. 요청 안에서만 재사용할 값은 지역 변수에 두고, 요청 간 캐시는 승인된 Redis 또는 별도 상태 저장 서비스에 보관한다.

```
const axios = require('axios');
const key = String($input).trim().toLowerCase();

const cached = await axios.get(`${$vars.CACHE_API_URL}/cache/${encodeURIComponent(key)}`, {
  headers: { Authorization: `Bearer ${$vars.CACHE_API_TOKEN}` },
  timeout: 2_000,
});
if (cached.status === 200 && cached.data?.value) {
  return cached.data.value;
}

return await computeAnswer(key);
```

캐시 service에는 TTL과 최대 entry 수를 설정한다. 캐시가 없어도 업무 처리가 가능해야 하며, 캐시 장애를 원본 처리 실패와 동일하게 취급하지 않는다.

7.7 파일 입력 처리

파일 입력 형식은 앞선 노드의 반환 형식과 일치시킨다. GenOS에서 공통으로 정하지 않은 문자열 앞부분을 기준으로 구분하지 않고 `type`과 `data`가 나뉜 객체를 사용한다.

```

const input = $vars.genosUploaded;

if (!input || typeof input !== "object") {
  throw new Error("genosUploaded must be an object");
}

if (input.type === "table" && Array.isArray(input.data)) {
  return { type: "table", rows: input.data };
}
if (input.type === "audio" && typeof input.transcript === "string") {
  return { type: "audio", transcript: input.transcript };
}
if (input.type === "text" && typeof input.text === "string") {
  return { type: "text", text: input.text };
}

throw new Error(`unsupported upload type: ${String(input.type)}`);

```

7.8 잘못된 작성 사례

```

// 잘못된 예 1) 시크릿 하드코딩
const token = "Bearer xoxb-..."; // PR 리뷰 차단

// 잘못된 예 2) sync blocking 외부 호출 (require 가능한 동기 라이브러리 사용 X)
const result = syncRequest(...); // 이벤트 루프 정지

// 잘못된 예 3) timeout 없는 외부 호출
await axios.post(url, data); // 무한 hang 가능

// 잘못된 예 4) globalThis에 요청 간 상태 저장
globalThis.cache[key] = value; // 다음 실행은 새 NodeVM이므로 cache hit 보장 없음

// 잘못된 예 5) JSON.parse 미검증
const data = JSON.parse($input); // 문자열 아니면 throw
// → typeof === 'string' 체크 후 parse

// 잘못된 예 6) GenOS 내부 서비스 직접 호출
await axios.post("http://workflow-pod-x:8080/run", ...); // gateway 우회
// → ${GENOS_URL}/api/gateway/workflow/... 경로 사용

// 잘못된 예 7) DB 커넥션 close 누락
const conn = await mysql.createConnection(...);
const [rows] = await conn.execute(...);
return rows; // conn.end() 누락 → leak

```

8. 전처리기 개발 가이드

8.1 기능 및 적용 범위

전처리기는 업로드 파일을 읽어 임베딩과 검색에 사용할 chunk 목록으로 바꾸는 Python 코드다. 문서 전처리 > 전처리기 > 전처리기 생성에서 제목, 상세 설명, 관리 그룹, 확장자, 전처리기 타입, 코드, 파라미터, GPU 할당량 및 환경 변수를 입력한다. 화면의 전처리기 타입은 첨부용 전처리기, 적재용 전처리기, 변환용 전처리기이며 내부 식별자는 각각 attachment, intelligent, convert다.

8.2 실행 형식

GenOS가 호출하는 클래스와 메서드는 다음 형식으로 작성한다.

```
from fastapi import Request

class DocumentProcessor:
    def __init__(self):
        pass

    async def __call__(
        self,
        request: Request,
        file_path: str,
        **kwargs: dict,
    ) -> list[dict]:
        ...
```

항목	작성 규칙
클래스명	DocumentProcessor로 작성한다.
생성자	GenOS가 DocumentProcessor()로 생성할 수 있도록 필수 인자를 두지 않는다.
호출 메서드	async def __call__(request, file_path, **kwargs)로 작성한다.
file_path	GenOS가 전달한 원본 파일 경로로 사용하고 원본을 덮어쓰지 않는다.
kwargs	전처리기 생성 화면의 파라미터에 등록된 키와 동일한 키를 읽는다. 내부 저장 항목은 default_params다.
반환값	list[dict]를 반환하며 각 dict의 text에는 임베딩할 문자열을 넣는다.

임베딩 단계는 각 chunk의 text 값을 직접 읽는다. 따라서 text가 없거나 문자열이 아니면 정상적인 전처리 결과로 사용할 수 없다. page, chunk 위치, bounding box 및 문서 속성은 검색 또는 화면 표시에서 실제로 사용하는 경우에만 별도 key로 추가한다.

8.3 기본 구현

다음 예시는 텍스트 파일을 하나의 chunk로 변환한다.

```

from pathlib import Path
from fastapi import Request

class DocumentProcessor:
    def __init__(self):
        pass

    async def __call__(
        self,
        request: Request,
        file_path: str,
        **kwargs: dict,
    ) -> list[dict]:
        path = Path(file_path)
        extension = path.suffix.lower()
        if extension != ".txt":
            raise ValueError(f"unsupported extension: {extension}")

        text = path.read_text(encoding="utf-8").strip()
        if not text:
            raise ValueError("document text is empty")

        return [{
            "text": text,
            "n_char": len(text),
            "n_word": len(text.split()),
            "n_line": len(text.splitlines()),
            "i_chunk_on_doc": 0,
            "n_chunk_of_doc": 1,
        }]

```

8.4 전처리 파라미터 작성

전처리기 생성 화면의 파라미터에 등록된 키와 코드의 kwargs 조회 키를 동일하게 유지한다. 숫자 파라미터는 사용 전에 변환하고 해당 전처리기가 지원하는 범위를 벗어나면 입력명을 포함한 오류를 발생시킨다.

```

chunk_size = int(kwargs.get("chunk_size", 1000))
chunk_overlap = int(kwargs.get("chunk_overlap", 100))

if not 100 <= chunk_size <= 5000:
    raise ValueError("chunk_size must be between 100 and 5000")
if not 0 <= chunk_overlap < chunk_size:
    raise ValueError("chunk_overlap must be smaller than chunk_size")

```

위 범위는 예시 전처리기에서 사용하는 값이다. 다른 범위를 사용하면 화면의 파라미터, 코드 및 전처리기 상세 설명에 같은 값을 적는다.

8.5 파일 처리 및 보안 규칙

- 외부 API 주소와 인증정보는 전처리기 생성·수정 화면의 환경 변수에 등록하고 `os.getenv()` 또는 `os.environ`으로 읽는다. 코드와 파라미터에 시크릿을 직접 입력하지 않는다.
- 전처리기에 등록된 지원 확장자와 코드가 실제로 처리하는 확장자를 일치시킨다.
- 확장자만으로 파일 형식을 판단하지 않는다. 형식별 parser를 호출하기 전에 파일 header를 확인한다.
- 변환 파일은 별도 임시 경로에 만들고 `finally`에서 삭제한다.

- 전달받은 원본 파일을 이름 변경, 이동 또는 덮어쓰기 하지 않는다.
- 로그에는 문서 원문, 개인정보 및 임시 파일 전체 경로를 기록하지 않는다.

```
from pathlib import Path
from tempfile import TemporaryDirectory

def ensure_pdf(path: Path) -> None:
    if path.suffix.lower() != ".pdf":
        raise ValueError("only PDF files are supported")
    with path.open("rb") as source:
        if source.read(5) != b"%PDF-":
            raise ValueError("file content is not PDF")

async def convert_and_parse(file_path: str) -> list[dict]:
    source = Path(file_path)
    ensure_pdf(source)

    with TemporaryDirectory(prefix="genos-preprocess-") as work_dir:
        converted = Path(work_dir) / "converted.pdf"
        await convert_file(source, converted)
        return await parse_pdf(converted)
```

parser 오류를 빈 목록으로 바꾸어 정상 처리로 보이게 하지 않는다.

전처리기에서 외부 OCR API 통신이 실패하면 05-00020001, OCR API가 실행 실패 응답을 반환하면 05-00020002를 로그에 기록한 뒤 예외를 발생시킨다. 지원하지 않는 파일 형식이나 parser 내부 오류에는 05-00020003을 기록한다. 오류 객체를 chunk 목록에 넣어 반환하지 않는다.

```
# 사용하지 않음
try:
    return await parse_document(file_path)
except Exception:
    return []

# 지원하지 않는 형식만 구분하고 parser 오류는 실패로 남긴다.
extension = Path(file_path).suffix.lower()
if extension not in {".pdf", ".txt"}:
    raise ValueError(f"unsupported extension: {extension}")
return await parse_document(file_path)
```

8.6 변경 및 확인

전처리기 저장 시 GenOS는 코드 길이를 최대 100만 자로 제한하고 Python 문법을 확인한다. 저장 성공은 실제 파일 처리 성공을 의미하지 않으므로 등록된 확장자별 대표 파일로 반환값을 확인한다.

전처리기별로 다음 테스트 파일을 준비한다.

테스트 파일	기대 결과
등록 확장자의 정상 파일	한 개 이상의 chunk 반환
빈 파일	빈 목록이 아닌 명시적 오류
손상된 파일	parser 오류 유지
지원하지 않는 확장자	지원 형식 오류
파라미터 최솟값·최댓값	정상 처리
범위 밖 파라미터	파라미터명을 포함한 오류

```
chunks = await processor(request, file_path, chunk_size=500)

assert len(chunks) >= 1
assert all(isinstance(chunk, dict) for chunk in chunks)
assert all(isinstance(chunk["text"], str) for chunk in chunks)
assert all(chunk["text"].strip() for chunk in chunks)
```

text는 임베딩 단계에서 직접 읽는 필수 key다. page, i_chunk_on_doc, 위치 좌표와 같은 부가 정보는 검색·인용·화면 표시 기능에서 실제로 사용하는 경우에만 항목과 형식을 정한다.

전처리 코드, 확장자 또는 파라미터가 변경되면 해당 전처리를 사용하는 AI Drive의 파일은 needs_reingest 상태로 표시된다. 이는 자동 재적재가 아니므로 변경 후 재적재를 실행하고 검색 결과에 새 chunk가 반영되었는지 확인한다.

1. 등록된 각 확장자의 대표 파일을 실행한다.
2. 반환값이 list[dict]이고 모든 항목의 text가 비어 있지 않은 문자열인지 확인한다.
3. page 또는 chunk 위치 정보를 추가했다면 첫 항목과 마지막 항목의 순번을 확인한다.
4. 실패 실행 후 임시 파일이 남지 않았는지 확인한다.
5. 전처리 본문 변경 후 AI Drive의 needs_reingest 상태를 확인하고 재적재한다.

9. 평가지표 개발 가이드

9.1 기능 및 실행 방식

평가지표는 평가 데이터셋의 각 행에 대해 evaluate(row, predict)를 실행하여 숫자 점수를 계산하는 Python 코드다. 평가 > 평가지표 > LLM 또는 평가 > 평가지표 > 워크플로우에서 평가지표 생성을 선택하고 제목, 상세 설명, 관리 그룹, 데이터셋, 데이터셋 리비전 및 코드를 입력한다. row에는 현재 데이터셋 행이 dict로 전달되고, predict는 평가 실행 시 선택한 LLM 또는 워크플로우를 호출하는 비동기 함수로 전달된다.

9.2 실행 형식

```
async def evaluate(row: dict, predict: callable) -> float:
    prediction = await predict(str(row["question"]))
    return 1.0 if prediction else 0.0
```

항목	작성 규칙
함수명	evaluate로 작성한다.
함수 형식	async def evaluate(row, predict)로 작성한다.
데이터셋 값	화면에서 선택한 데이터셋 리비전의 column 이름으로 row["column_name"]에서 읽는다.
예측 호출	await predict(question)으로 호출한다.
반환값	float()로 변환할 수 있는 숫자를 반환한다. None이나 오류 문자열을 반환하지 않는다.

GenOS 실행 환경은 동기 evaluate 함수와 await가 없는 predict 호출을 일부 보정한다. 새 평가지표는 위 형식에 맞게 코드를 작성한다.

9.3 점수 계산 예시

다음 예시는 예측 결과에 기준 키워드가 포함된 비율을 0에서 1 사이 점수로 반환한다.

평가지표를 작성하기 전에 다음 내용을 상세 설명과 테스트에 동일하게 적는다.

항목	예시
필수 column	question, expected_keywords
점수 범위	0.0~1.0
좋은 점수 방향	높을수록 좋음
계산식	일치한 keyword 수 / 전체 expected keyword 수
계산 불가 행	ValueError 발생

GenOS는 점수 범위와 방향을 자동으로 검사하지 않는다. 따라서 평가지표를 사용하는 사람이 점수를 같은 의미로 이해할 수 있도록 상세 설명에 직접 적는다.

```

async def evaluate(row: dict, predict: callable) -> float:
    """예측 결과에 포함된 기준 키워드 비율을 반환한다."""
    required_columns = {"question", "expected_keywords"}
    missing = required_columns - row.keys()
    if missing:
        raise ValueError(f"missing dataset columns: {sorted(missing)}")

    expected_keywords = [
        str(keyword).strip().lower()
        for keyword in row["expected_keywords"]
        if str(keyword).strip()
    ]
    if not expected_keywords:
        raise ValueError("expected_keywords must not be empty")

    prediction = str(await predict(str(row["question"]))).lower()
    matched = sum(
        1 for keyword in expected_keywords
        if keyword in prediction
    )
    return matched / len(expected_keywords)

```

GenOS는 점수를 `float()`로 변환하지만 점수 범위를 정하지 않는다. 평가지표별로 0~1, 백분율 또는 기타 점수 범위 중 하나를 정하고 상세 설명과 코드에서 같은 값을 사용한다. 서로 다른 평가지표 결과를 함께 비교할 때는 점수가 커지는 방향과 범위를 같게 변환한다.

9.4 데이터셋 및 예측 호출 규칙

- 필수 `column`은 `row.get()`의 빈 기본값으로 숨기지 않고 누락된 `column` 이름을 포함한 예외로 남긴다.
- 하나의 행을 평가하는 동안 같은 입력에 대한 `predict`를 반복 호출하지 않는다. 결과를 지역 변수에 저장해 점수 계산에 재사용한다.
- 점수 계산 실패를 0.0으로 바꾸지 않는다. 실제 0점과 실행 오류를 구분할 수 없기 때문이다.
- 입력 질문, 예측 전문 및 개인정보를 일반 로그에 기록하지 않는다. 오류 로그에는 평가 행 번호와 실패 단계처럼 원인 추적에 필요한 값만 남긴다.

```
# 사용하지 않음: 같은 행에서 예측을 두 번 실행한다.
if await predict(question):
    return score(await predict(question))

# 한 번 실행한 결과를 재사용한다.
prediction = await predict(question)
return score(prediction)
```

9.5 오류 처리

전체 평가 실행은 오류가 발생한 행을 오류 건수에 반영하고 다음 행 평가를 계속한다. 따라서 평가지표 코드는 광범위한 `except Exception`으로 실패를 숨기지 않고 필수 `column` 누락, 빈 기준값 및 계산 불가 상태를 구체적인 예외로 남긴다.

`predict` 호출의 통신 실패를 직접 확인하여 처리하는 경우에는 06-00020001, `predict`의 실행 실패 응답을 처리하는 경우에는 06-00020002를 로그에 기록한다. 필수 `column` 누락, 점수 변환 실패 및 계산 오류에는 06-00020003을 기록하고 예외를 발생시킨다. 오류 코드나 오류 메시지를 평가지표의 점수로 반환하지 않는다.

```
# 사용하지 않음
try:
    return calculate_score(row)
except Exception:
    return 0.0

# 계산할 수 없는 입력은 오류로 구분한다.
reference = str(row["reference"]).strip()
if not reference:
    raise ValueError("reference must not be empty")
return calculate_score(reference)
```

9.6 실행 및 확인

평가지표 테스트는 데이터셋에서 최대 3개 행을 실행한다. LLM 평가지표에서는 서빙 API, 워크플로우 평가지표에서는 워크플로우를 선택한 뒤 테스트를 실행하고 결과를 확인한다. 테스트 통과 후 전체 평가를 실행하고 오류 행 수와 점수 분포를 확인한다.

- 평가지표에 연결한 데이터셋 리버전의 `column` 이름과 자료형을 확인한다.
- 정상 행, 필수 `column` 누락 행 및 빈 기준값 행으로 평가지표 테스트를 실행한다.
- 각 정상 결과가 숫자로 반환되고 문서화된 점수 범위와 일치하는지 확인한다.
- 전체 평가에서 오류 행이 발생하면 해당 행의 `column`과 `predict` 호출 실패 여부를 확인한다.

정상 테스트 데이터에서는 `predict`가 행당 한 번만 호출되고 결과가 정한 점수 범위에 포함되는지 확인한다.

```

async def test_keyword_recall():
    calls = 0

    async def predict(question: str) -> str:
        nonlocal calls
        calls += 1
        return "카드 분실 시 앱에서 즉시 정지할 수 있습니다."

    score = await evaluate(
        row={
            "question": "카드를 분실했어요",
            "expected_keywords": ["분실", "정지"],
        },
        predict=predict,
    )

    assert calls == 1
    assert 0.0 <= float(score) <= 1.0

```

필수 column 누락은 실제 0점으로 변환하지 않는다.

```

with pytest.raises(ValueError, match="expected_keywords"):
    await evaluate(
        row={"question": "카드를 분실했어요"},
        predict=predict,
    )

```

10. GenOS 서비스 연계 및 재사용

Agent, Tool, LLM, Prompt는 GenOS에서 제공하는 표준 경로로 호출한다. 같은 기능을 여러 영역에서 서로 다른 방법으로 호출하면 인증, 실행 기록 및 호출 횟수 제한이 빠질 수 있다.

여러 곳에서 함께 사용하는 모듈, 공통 함수 및 패키지는 제10장의 재사용 기준을 따른다.

연계 항목	사용 방법	금지 사항
Agent 실행	Workflow Python Step·Flowise 노드로 구성하거나 코드 서빙 애플리케이션으로 구현하고, 필요한 경우 A2A로 외부에 노출	목적 없이 반복 횟수가 제한되지 않은 agent loop 구현
Tool 호출	MCP tools/call (/api/gateway/mcp/<id>/mcp)	K8s service 직접 호출
LLM 호출	GenOS Gateway의 OpenAI 호환 경로 (/api/gateway/rep/serving/<id>/v1/chat/completions)	관리 대상 모델을 외부 SDK와 별도 자격증명으로 우회 호출
Prompt 로딩	Workflow의 MNC Prompt 노드 또는 admin-api의 /prompt/template/<id>	코드 안 긴 문자열 인라인 / 코드 PR로 프롬프트 변경

10.1 영역별 표준 호출 적용

10.1.1 MCP 도구 (INTERNAL_PYTHON)

- **GenOS 표준:** 이 MCP 안에서 GenOS 관리형 LLM(10.2절)·다른 MCP 도구(10.3절)·프롬프트(10.5절)를 호출할 때는 제10장 GenOS 표준 호출 경로를 사용한다.
- **공통 모듈:** 만든 `@mcp.tool` 함수를 여러 MCP 서버에서 쓰려면 10.8절 Tool 레벨 패턴 A·B, 도구 묶음을 여러 워크플로우에서 쓰려면 10.8절 Server 레벨.

10.1.2 워크플로우 Python 단계

- **GenOS 표준:** Step 안에서 LLM·다른 도구·프롬프트 호출은 제10장 GenOS 표준 호출 경로 — `call_llm`(10.2절) / `call_mcp_tool`(10.3절) / `load_prompt`(10.5절).
- **공통 모듈:** 만든 Workflow를 다른 Agent가 Tool처럼 호출하게 하려면 10.9절의 A2A 카드 노출 방법을 사용한다. 여러 하위 Workflow 중 호출 대상을 선택하는 Master Agent는 10.9절을 따른다.
- **헬퍼 추출:** 같은 입력 형식과 변경 주기를 공유하는 step이 여러 곳에서 사용되면 10.10절의 공통 모듈 분리를 검토한다.

10.1.3 코드 서빙

- **GenOS 표준:** 사용자 웹 애플리케이션에서 LLM과 다른 MCP 도구는 Gateway 경로로 호출하고, 프롬프트는 10.5절의 `admin-api /prompt/template/<id>` 경로에서 불러온다.
- **공통 모듈:** 같은 애플리케이션을 MCP 처리 서버로도 연결하려면 6.5절의 `/mcp/list`와 `/mcp/call` 요청·응답 형식을 구현하고 등록 화면의 호출 경로와 일치시킨다. Python 함수 이름이 정해져 있다고 가정하지 않는다.

10.1.4 JavaScript/TypeScript MCP

- **GenOS 표준:** TS 도구 안에서 GenOS 관리형 LLM을 호출할 때는 10.2절의 `fetch({$GENOS_URL}/api/gateway/rep/serving/<id>/v1/chat/completions, ...)` 패턴을 사용한다. 외부 SDK 직접 호출을 허용할지는 고객사 자격증명·감사 정책으로 별도 결정한다.
- **공통 모듈:** 도구 함수를 npm 패키지로 publish 해 여러 MCP 서버에서 import 하는 패턴은 10.8절 패턴 A — Verdaccio 사내 레지스트리 (10.7절).

10.1.5 Spring AI MCP

- **GenOS 표준:** `@Tool` 메서드 안에서 GenOS 관리형 LLM·다른 도구·프롬프트를 호출할 때는 `RestTemplate` 또는 `WebClient`로 제10장 GenOS 표준 호출 경로를 사용한다. 외부 키를 사용하는 별도 `ChatClient`는 고객사 보안정책에서 허용한 경우에만 구성한다.
- **공통 모듈:** 공용 클라이언트, 입력 형식 및 인증 함수는 사내 Maven 모듈 또는 JAR로 분리하여 여러 Spring AI MCP에서 사용한다(10.7절). 여러 워크플로우가 같은 MCP 서버를 사용하는 방법은 10.8절 패턴 B를 따른다.

10.1.6 Flowise Custom JS Function

- **GenOS 표준:** CustomFunction에서 LLM, MCP 및 워크플로우는 각 Gateway 경로로 호출한다. 프롬프트는 직접 HTTP 호출보다 Prompts > MNC Prompt 노드로 선택하여 연결한다. 자격증명은 코드에 직접 입력하지 않고 `$vars`로 주입한다.

- **공통 모듈:** 같은 JavaScript 코드가 여러 워크플로우에서 반복되면 MCP 서버나 A2A Workflow로 분리한다(10.8절 패턴 B, 10.9절). CustomFunction에는 간단한 결과 변환과 항목 연결만 작성한다.

10.2 LLM 호출

표준 경로

GenOS가 관리하는 LLM은 Gateway의 OpenAI 호환 경로로 호출한다. Gateway를 사용하면 모델별 인증정보, 호출 경로, 호출 횟수 제한 및 실행 기록을 한 곳에서 관리할 수 있다.

직접 배포한 모델은 vLLM 등의 모델 서버가 추론을 수행한다. 외부 API 서빙 가운데 별도의 변환이 필요한 유형은 Gateway 내부에서 LiteLLM을 거쳐 공급자 API로 전달될 수 있다. LiteLLM은 모델을 실행하는 서버가 아니라 외부 모델의 요청 형식, 인증정보 및 호출 경로를 연결하는 프록시이므로 사용자 코드는 LiteLLM 주소를 직접 호출하지 않고 위의 Gateway 경로만 사용한다.

```
# Python 공통 함수 – MCP INTERNAL_PYTHON / Workflow Python Step / Python Code Serving
import os, httpx

async def call_llm(messages: list[dict], model: str, stream: bool = False):
    url = f"
{os.environ['GENOS_URL']}/api/gateway/rep/serving/{os.environ['LLM_SERVING_ID']}/v1/chat/completions"
    headers = {"Authorization": f"Bearer {os.environ['GENOS_TOKEN']}"}
    body = {"model": model, "messages": messages, "stream": stream}
    async with httpx.AsyncClient(timeout=60.0) as c:
        r = await c.post(url, headers=headers, json=body)
        r.raise_for_status()
        return r.json()
```

```
// JS/TS MCP – Node fetch 패턴
const r = await fetch(

`${process.env.GENOS_URL}/api/gateway/rep/serving/${process.env.LLM_SERVING_ID}/v1/chat/completions`,
{
    method: "POST",
    headers: { Authorization: `Bearer ${process.env.GENOS_TOKEN}`, "Content-Type":
"application/json" },
    body: JSON.stringify({ model: process.env.LLM_MODEL_ID, messages, stream: false }),
    signal: AbortSignal.timeout(60_000),
},
);
```

금지

- GenOS 관리 대상 모델을 별도 외부 키와 SDK로 우회 호출하지 않는다. 직접 SDK 호출 금지가 고객사 보안정책인 경우 외부 모델에도 동일하게 적용한다.
- GenOS 내부 라우팅 헤더(x-genos-litellm-model, x-genos-body-* 등)를 사용자 코드에서 직접 작성하지 않는다. 요청 본문에는 호출 대상이 지원하는 OpenAI 호환 필드를 사용한다.

작성 규칙

- timeout은 3.6절의 전체 처리 시간에 맞춘다. 실시간 전송은 최초 연결 시간과 전체 전송 시간을 나누어 정한다.
- 실시간 전송: stream=True로 호출하고 SSE 응답을 읽는다. Workflow Python Step은 yield {"event": "token", "data": chunk}로 전달하고, Code Serving은 사용하는 웹 프레임워크의 실시간 응답 기능을 사용한다.
- 다시 시도할 리비전과 대신 사용할 리비전은 서빙의 라우팅 정책에서 설정한다. 사용자 코드에 횟수 제한이 없는 재시도 반복문을 작성하지 않는다.

10.3 Tool 호출

표준 경로

도구는 **MCP tools/call JSON-RPC**로 호출한다. 도구 함수나 서비스를 직접 연결하여 호출하는 방식을 금지한다.

```
# Python 공통 함수 - MCP / Workflow Python Step / Python Code Serving
async def call_mcp_tool(server_id: int, tool_name: str, arguments: dict) -> dict:
    url = f"{os.environ['GENOS_URL']}/api/gateway/mcp/{server_id}/mcp"
    headers = {"Authorization": f"Bearer {os.environ['GENOS_TOKEN']}"}
    body = {
        "jsonrpc": "2.0", "id": 1, "method": "tools/call",
        "params": {"name": tool_name, "arguments": arguments},
    }
    async with httpx.AsyncClient(timeout=30.0) as c:
        r = await c.post(url, headers=headers, json=body)
        r.raise_for_status()
        return r.json()["result"]
```

Workflow에서는 **Flowise MCP 노드**를 먼저 사용한다. 분기 또는 재호출 때문에 노드를 사용하기 어려울 때만 Python Step에서 공통 함수를 사용한다.

금지

- MCP Pod의 K8s Service DNS를 직접 호출하지 않는다. 인증과 실행 추적이 빠지므로 반드시 `/api/gateway/mcp/...`를 사용한다.
- 동일한 업무 기능을 여러 영역에서 함께 변경해야 한다면 기존 구현을 한 곳에서 관리하고 나머지 영역에서 호출하는 구조를 우선 검토한다. 영역별 실행 방식이나 성능 요구가 다르다면 별도 구현할 수 있으며, 그 사유를 코드 설명에 기록한다.

10.4 Agent 실행

Agent는 Workflow의 Python Step과 Flowise 노드를 연결하여 구성하거나 코드 서빙 애플리케이션으로 구현한다. 다른 에이전트에 공개할 때는 A2A를 사용한다. 코드 서빙 에이전트의 구성은 6.10절을 따른다.

Pattern A — Python Step 으로 워크플로우 조립

여러 Python step을 연결하여 의도 파악, 도구 호출, 결과 합성을 나누어 처리한다. step 간 상태는 data dict에 누적한다. GenOS 정보는 `data['genos_state']`에 자동으로 추가된다(5.3절).

```
[Step1 의도파악] → [Step2 Tool 호출] → [Step3 결과합성·스트리밍]
call_llm(...)      call_mcp_tool(...)  async for chunk in stream_llm(...)
```

```
# Step1 - 의도 파악
async def run(data: dict) -> dict:
    decision = await call_llm(
        [{"role": "system", "content": "사용자 의도를 한 단어로 분류"},
         {"role": "user", "content": data["question"]}],
        model=os.environ["LLM_MODEL_ID"],
    )
    data["intent"] = decision["choices"][0]["message"]["content"]
    return data
```

```
# Step2 - MCP 도구 호출
async def run(data: dict) -> dict:
    if data["intent"] == "search":
        data["search_result"] = await call_mcp_tool(
            server_id=int(os.environ["SEARCH_MCP_ID"]),
            tool_name="web_search",
            arguments={"query": data["question"]},
        )
    return data
```

```
# Step3 - 프롬프트 로딩 + 스트리밍 합성
async def run(data: dict):
    prompt = await load_prompt(int(os.environ["ANSWER_PROMPT_ID"]))
    messages = [
        {"role": "system", "content": prompt.format(context=data.get("search_result", ""))},
        {"role": "user", "content": data["question"]},
    ]
    accumulated = ""
    async for chunk in stream_llm(messages):
        accumulated += chunk
        yield {"event": "token", "data": chunk}
    yield {"event": "result", "data": {**data, "text": accumulated}}
```

일반 Workflow에서 data['genos_state']['session_id']는 같은 대화에 속한 요청을 구분하는 값이다. Python Step의 이전 대화를 자동으로 저장하거나 data에 넣어주지는 않으므로, 이전 대화가 필요하다면 애플리케이션에서 저장하고 불러오는 방법을 정한다.

Pattern B — A2A 로 워크플로우 노출

A2A는 작성한 워크플로우를 다른 에이전트나 외부 시스템이 하나의 에이전트처럼 호출하도록 공개하는 방식이다. 일반 Workflow 호출에 Agent Card, Task 상태, 대화 이력, Artifact 및 실시간 전송 형식을 추가한다.

GenOS에서 A2A 에이전트로 노출을 설정하고 워크플로우를 재배포하면 Agent Card와 호출 주소가 생성되거나 갱신된다. Gateway는 agent_card_id를 실제 Workflow로 연결하고, Workflow Pod는 Task를 생성하거나 기존 Task를 이어서 내부 /run/v2를 실행한다. 기존 task_id가 전달되면 최근 메시지를 Flowise history로 바꾸고 같은 값을 chatId로 사용한다. 실행 메시지와 상태 및 Artifact는 Task 이력으로 저장한다.

여러 Workflow를 함께 사용할 때는 각 Workflow를 A2A 에이전트로 노출하고, Master Agent가 Agent Card의 에이전트 설명과 자동 구성된 보유 기술을 보고 호출 대상을 선택한다. 선택한 Agent에는 tasks/send 또는 tasks/sendSubscribe로 요청하고, 후속 입력에는 기존 task_id를 전달하여 같은 대화를 이어 간다.

```
[외부 Agent / 다른 GenOS Workflow]
| POST /a2a/<agent_card_id>
| JSON-RPC method: tasks/send
▼
[GenOS Workflow (Pattern A 결과물)]
|
|─ token 스트리밍 (yield {"event": "token", "data": chunk})
|─ input-required (5.4절 의 3채널 중 택1)
|─ 최종 결과 (yield {"event": "result", "data": dict})
```

- 워크플로우의 **A2A 에이전트 설정**에서 **A2A 에이전트로 노출**을 선택하고 에이전트 이름, 에이전트 설명, 지원 기능을 입력한 뒤 워크플로우를 재배포한다. 재배포하면 Agent Card와 표준 A2A 엔드포인트가 생성되거나 갱신된다.

- A2A 에이전트 상세 화면의 보유 기술은 직접 편집하는 항목이 아니다. 워크플로우를 MCP 도구로 노출한 경우 해당 도구의 이름과 설명으로 자동 구성되며, 그렇지 않으면 빈 목록으로 표시될 수 있다.
- 일반 호출은 POST /a2a/<agent_card_id>에 JSON-RPC tasks/send를 전송한다. 실시간 호출은 POST /a2a/<agent_card_id>/stream에 tasks/sendSubscribe 요청을 전송한다.
- input-required는 5.4절의 세 가지 방법 중 하나로 전달한다. status 이벤트, result의 a2a_status, 단순 return을 사용할 수 있다.
- 일반 호출과 실시간 호출 모두 params.id에 기존 task_id를 입력하면 저장된 최근 메시지를 불러와 같은 Task를 이어서 처리한다.
- 일반 Workflow의 session_id와 A2A의 task_id는 용도가 다르므로 서로 대신 사용하지 않는다.

작성 규칙

- 자체 agent loop가 필요한 경우 최대 반복 횟수, 허용 Tool 목록, Tool 입력 확인 및 종료 조건을 코드에 명시한다. 단계별 확인과 운영 추적이 중요하면 여러 Python step 또는 Flowise Agentflow 노드로 나눈다.
- 다른 워크플로우, Code Serving 또는 MCP를 K8s service DNS로 직접 호출하지 않는다. 반드시 /api/gateway/... 경로를 사용한다.

10.5 Prompt 로딩

표준 경로

프롬프트는 **GenOS Prompt 리소스**에 등록하고 ID로 참조한다. 도구 > 프롬프트 라이브러리에서 프롬프트를 생성하고 리비전을 관리한다.

Prompt Library의 실행용 본문은 Gateway가 아니라 admin-api의 다음 경로에서 조회한다.

```
GET /prompt/template/{prompt_id}
```

클러스터 내부에서는 http://llmops-admin-api-service:8080/prompt/template/{prompt_id}, 클러스터 외부에서는 https://<GenOS-host>/api/admin/prompt/template/{prompt_id} 형식으로 호출한다. /api/gateway/prompt/... 경로는 사용하지 않는다.

```
import os
import httpx

async def load_prompt(prompt_id: int) -> str:
    admin_api_url = os.environ["GENOS_ADMIN_API_URL"].rstrip("/")
    url = f"{admin_api_url}/prompt/template/{prompt_id}"

    async with httpx.AsyncClient(timeout=10.0) as c:
        r = await c.get(url)
        r.raise_for_status()

        payload = r.json()
        if payload.get("code") != 0:
            raise RuntimeError(payload.get("errMsg", "프롬프트 조회 실패"))
        return payload["data"]
```

코드 서빙에서 호출할 때는 코드 서빙 리비전 상세 > 환경 설정 > 환경 변수에 GENOS_ADMIN_API_URL을 등록한다. 클러스터 내부 값은 http://llmops-admin-api-service:8080, 외부 호출 값은 https://<GenOS-host>/api/admin 형식을 사용한다.

Workflow에서는 Flowise의 Prompts > MNC Prompt 노드를 추가하고 Prompt Template에서 프롬프트를 선택한다. 이 노드는 실행 시 `/prompt/template/{prompt_id}`에서 운영 적용된 본문을 불러오고 `{question}`, `{context}`와 같은 항목을 PromptTemplate 입력 변수로 연결한다.

금지

- 코드 내부에 긴 프롬프트 문자열을 직접 작성하지 않는다. 변경 시 코드 반영 절차가 필요하며 비개발자가 수정하기 어렵다. 단문 시스템 메시지는 예외로 할 수 있다.
- 프롬프트 ID를 코드에 직접 입력하지 않는다. `ANSWER_PROMPT_ID`, `INTENT_PROMPT_ID` 등의 환경변수로 전달한다.

작성 규칙

- 다국어 프롬프트는 언어별로 별도 ID를 등록하고 호출 코드에서 사용자의 locale에 따라 선택한다.
- 운영 환경별로 프롬프트를 분리해야 하면 dev, stg, prod에 별도 ID를 등록하고 환경변수로 전달한다.
- `/prompt/template/{prompt_id}`는 운영 적용된 리비전의 본문을 반환한다. 새 리비전을 만든 뒤 운영에 반영해야 실행 코드와 MNC Prompt 노드에서 새 본문을 조회할 수 있다.
- 운영 적용된 프롬프트 본문만 변경한 경우 같은 `prompt_id`를 사용하는 호출 코드를 수정하지 않는다.

10.6 재사용 구조

제10장은 표준 호출 경로와 재사용 단위를 함께 정의한다. 같은 기능을 여러 MCP·워크플로우에 산발 구현하지 않으려면 **한 번만 들면 N번 재사용** 원칙으로 단위를 쌓아 올린다.

```
[ 사내 패키지 (PyPI · npm · JAR) ]
  | import
  ▼
[ MCP Tool - @mcp.tool 함수 1개 ]
  | 같은 도구를 여러 MCP 서버에서 import
  ▼
[ MCP Server - 여러 Tool의 묶음 ]
  | 같은 server_id를 여러 워크플로우에서 참조
  ▼
[ Workflow - Step 체이닝 = 단일 에이전트 ]
  | A2A 에이전트로 호출 → 다른 에이전트의 도구처럼 호출
  ▼
[ Master Agent - 여러 A2A Workflow 중 호출 대상 선택 ]
```

여러 구현이 같은 입력 형식과 변경 주기를 공유하면 한 단계 아래의 함수·패키지·MCP 서버 단위로 분리하여 재사용할 수 있다.

10.7 패키지 수준 재사용

GenOS 사내에 다음 3개 레지스트리가 운영된다.

언어	레지스트리	파일 형식	등록 위치
Python	PyPI 서버 (NFS 기반)	.whl	관리 > 리소스 > PyPI 패키지
JS/TS	Verdaccio (llmops-npm-registry-service:4873)	.tgz	관리 > 리소스 > NPM 패키지에서 확인하고 <code>npm publish --registry ...</code> 로 게시
Java	NFS JAR 저장소	.jar	관리 > 리소스 > JAR 패키지

패키지에 담을 후보

- `call_llm`, `call_mcp_tool`, `load_prompt` 등 제10장의 표준 호출을 처리하는 공통 함수
- 공통 Pydantic 모델(Tool 입력·출력 항목)
- 공통 인증·로깅·trace wrapper
- 사내 도메인 클라이언트 (사내 API, ERP, etc.)

```
# 사내 PyPI 패키지 예: genos-common 0.3.0
# genos_common/llm.py
import os, httpx

async def call_llm(messages, model, stream=False, timeout=60.0):
    url = f"
{os.environ['GENOS_URL']}/api/gateway/rep/serving/{os.environ['LLM_SERVING_ID']}/v1/chat/completions"
    headers = {"Authorization": f"Bearer {os.environ['GENOS_TOKEN']}"}
    body = {"model": model, "messages": messages, "stream": stream}
    async with httpx.AsyncClient(timeout=timeout) as c:
        r = await c.post(url, headers=headers, json=body)
        r.raise_for_status()
        return r.json()
```

```
# 사용 측 - MCP / Workflow Python Step / Python Code Serving
from genos_common.llm import call_llm
from genos_common.mcp import call_mcp_tool
from genos_common.prompt import load_prompt
```

작성 규칙

- 패키지 이름만 보고 사용 업무와 역할을 알 수 있도록 작성한다. 예: `genos-common`, `crm-tools`, `finance-agents`. 이 이름 형식은 사내 패키지 작성 기준이다.
- 패키지 버전과 이전 버전 지원 기준은 사내 저장소 운영 기준을 따른다. GenOS는 특정 버전 작성 방식이나 사용 중단 기간을 정하지 않는다.
- 패키지가 GenOS 환경에 의존하면 README 첫 줄에 필요한 환경변수를 명시한다. 예: `requires GENOS_URL, GENOS_TOKEN, LLM_SERVING_ID`.
- 패키지 안에 시크릿의 기본값을 직접 입력하지 않는다. 필수 환경변수가 없으면 즉시 예외를 발생시킨다.

10.8 Tool 및 MCP Server 재사용

10.8.1 Tool 수준 재사용

같은 도구를 여러 곳에서 사용할 때는 다음 두 가지 방식 중 개발 환경에 맞는 방식을 선택한다.

패턴 A — Tool 함수를 패키지로 분리하여 여러 MCP 서버에서 사용

```
# 패키지 my-tools
# my_tools/search.py
async def web_search(query: str, limit: int = 5) -> str:
    """웹 검색 결과를 JSON 문자열로 반환한다."""
    ...
```

```
# MCP 서버 A
from my_tools.search import web_search

mcp.tool()(web_search)          # GenOS가 제공한 mcp 인스턴스에 동일 함수 등록
```

```
# MCP 서버 B - 같은 도구 + 추가 도구
from my_tools.search import web_search
from my_tools.translate import translate

mcp.tool()(web_search)
mcp.tool()(translate)
```

도구의 처리 내용을 변경할 때는 공통 패키지를 수정하고 새 버전을 게시한 후 해당 패키지를 사용하는 MCP 서버의 의존성을 갱신한다.

패턴 B — 여러 워크플로우가 같은 MCP 서버 사용

Tool 함수를 별도 패키지로 분리하지 않고, **MCP 서버 한 개를 만들어 여러 워크플로우가 같은 server_id를 사용**하도록 한다. 패키지를 별도로 관리하지 않아도 되어 설정이 단순하다.



워크플로우 등록 화면에서 MCP 서버를 선택하면 해당 서버의 도구가 자동으로 표시된다. 도구를 변경할 때는 MCP 서버를 수정하고 다시 배포한다.

선택 기준

- **패턴 A:** 도구 함수에 사내 라이브러리, 로컬 캐시 또는 여러 의존성이 필요할 때 사용한다. 도구 함수를 패키지로 분리하면 개별적으로 빌드하고 테스트할 수 있다.
- **패턴 B:** 도구가 외부 API를 호출하는 수준의 단순한 함수일 때 사용한다. 하나의 서버를 여러 워크플로우에서 참조하도록 구성할 수 있다.

10.8.2 MCP Server 수준 재사용

작성 규칙

- MCP 서버는 함께 배포하고 같은 권한·변경 주기로 관리할 Tool을 묶는다. 예: `customer-service-tools`, `data-analytics-tools`. `description`만으로 용도를 구분하기 어려운 Tool은 서버를 나눈다.
- 서버의 Tool 수는 고정하지 않는다. 서로 다른 업무의 Tool이 섞여 `description`만으로 용도를 구분하기 어려우면 서버를 나눈다.
- 도구 이름은 3.1절의 `{domain}_{action}_{resource}[_{qualifier}]` 형식을 사용한다.
- 여러 코드에서 같은 입력·출력 형식을 사용해야 하면 필드명·타입·필수 여부를 먼저 정한 후 공통 패키지 사용 여부를 결정한다. GenOS가 특정 형식을 자동으로 적용하지는 않는다.

10.9 Workflow 및 Master Agent 재사용

10.9.1 Workflow 수준 재사용

워크플로우는 여러 Python Step, MCP 호출, LLM 결과 합성 및 Prompt 불러오기를 조합하여 작성한다. 구성 방식은 10.4절의 Pattern A와 같다.

워크플로우를 A2A 카드로 노출 — 도구화

작성한 워크플로우를 다른 에이전트가 도구처럼 호출하게 하려면 워크플로우의 **A2A 에이전트 설정**에서 **A2A 에이전트로 노출**을 선택한다.

- 설정을 저장하고 워크플로우를 재배포하면 A2A Agent Card와 표준 호출 경로(/a2a/<agent_card_id>)가 생성되거나 갱신된다.
- 다른 Workflow, Master Agent 및 외부 시스템이 같은 방법으로 호출한다.

```
[Master Agent]
| POST /a2a/<agent_card_id>
| JSON-RPC method: tasks/send
▼
[하위 Workflow A (A2A 카드)] ← 여러 Master가 같은 Workflow 호출
```

작성 규칙

- **A2A Agent Card 설명** — 에이전트 이름, 에이전트 설명, 지원 기능에 에이전트의 용도와 호출 특성을 작성한다. 보유 기술은 워크플로우의 MCP 도구 노출 설정에서 가져오므로 상세 화면에서 직접 작성하지 않는다. 설정을 저장한 뒤 워크플로우를 재배포하여 Agent Card를 갱신한다.
- **호출 식별값 전달** — 서로 관련된 새 Task를 묶을 때는 contextId를 전달한다. 기존 Task를 이어서 처리할 때는 JSON-RPC params.id에 기존 task_id를 전달한다. 일반 Workflow의 session_id를 A2A task_id 대신 사용하지 않는다.
- **input-required 전달** — 하위 Workflow가 input-required를 반환하면 Master도 5.4절의 같은 방법으로 상위에 전달한다.

10.9.2 Master Agent 수준 재사용

Master는 **요청 분류** → **하위 Workflow 호출** → **결과 정리**를 담당하도록 구성할 수 있다. GenOS가 이 역할을 강제로 나누지는 않으므로, Master가 직접 처리할 기능과 하위 Workflow가 처리할 기능을 프로젝트에서 정한다.

```
# Master Workflow Python Step (요청 분류 + 호출 대상 선택)
from genos_common.llm import call_llm
from genos_common.a2a import call_a2a_agent

SUB_AGENT_MAP = {
    # 환경변수로 설정할 수 있음
    "customer": int(os.environ["CUSTOMER_AGENT_ID"]),
    "billing": int(os.environ["BILLING_AGENT_ID"]),
    "general": int(os.environ["GENERAL_AGENT_ID"]),
}

async def run(data: dict):
    intent = await classify_intent(data["question"]) # call_llm 으로 분류
    sub_id = SUB_AGENT_MAP.get(intent, SUB_AGENT_MAP["general"])
    result = await call_a2a_agent(
        agent_card_id=sub_id,
        question=data["question"],
        context_id=data["genos_state"]["session_id"],
    )
    yield {"event": "result", "data": {**data, "sub_agent_result": result}}
```

```
# genos_common.a2a - 공통 패키지에 작성
import os, uuid, httpx

async def call_a2a_agent(
    agent_card_id: int,
    question: str,
    context_id: str,
    task_id: str | None = None,
):
    url = f"{os.environ['GENOS_URL']}/a2a/{agent_card_id}"
    params = {
        "contextId": context_id,
        "message": {
            "role": "user",
            "parts": [{"type": "text", "text": question}],
        },
    }
    if task_id:
        params["id"] = task_id

    body = {
        "jsonrpc": "2.0",
        "id": str(uuid.uuid4()),
        "method": "tasks/send",
        "params": params,
    }
    async with httpx.AsyncClient(timeout=120.0) as c:
        r = await c.post(url, json=body)
        r.raise_for_status()
        payload = r.json()
        if "error" in payload:
            raise RuntimeError(payload["error"]["message"])
        return payload["result"]
```

작성 규칙

- **하위 Workflow의 A2A Agent Card**는 에이전트 설명과 자동 구성된 보유 기술을 호출 대상 선택에 사용한다. 하위 Workflow가 추가되면 연결 환경변수와 대상 선택 정보를 함께 갱신한다.
- **Master 역할** — 호출 대상 선택과 직접 처리할 업무의 범위를 프로젝트에서 정하고, Agent 구성 문서와 코드에 같은 내용을 적는다.
- **하위 Workflow 호출 시간** — 호출 코드의 제한 시간과 전체 처리 시간은 3.6절의 기준에 따라 정한다. 실시간 응답이 필요하다면 /a2a/<agent_card_id>/stream을 사용한다.

10.10 공통 모듈 및 GenOS 리소스 공유

10.10.1 횡단 공통 모듈

여러 업무에서 공통으로 사용하는 기능은 별도 모듈로 분리할 수 있다.

후보	추천 위치	효과
call_llm / stream_llm	genos-common.llm	10.2절의 표준 호출과 재시도·대체 모델 설정을 한 곳에서 처리
call_mcp_tool	genos-common.mcp	10.3절 JSON-RPC 페이로드 표준
load_prompt (LRU 캐시)	genos-common.prompt	10.5절 GenOS Prompt 리소스 + 로컬 캐시
call_a2a_agent	genos-common.a2a	하위 Agent 호출 + input-required 전파
공통 Pydantic 모델 (ToolInput, ToolOutput, Citation)	genos-common.schema	Tool 사이의 입력·출력 항목 통일
로그·실행 추적 함수	genos-common.observability	trace_id 자동 추가, extra= 항목 통일
토큰 발급/갱신	genos-common.auth	만료 자동 재발급, 시크릿 외부 노출 차단
공통 테스트 데이터 / 가상 LLM / 가상 MCP	genos-common.testing	같은 방법으로 단위 테스트 수행

10.10.2 GenOS 리소스 공유

코드를 별도 패키지로 분리하지 않고도 **GenOS가 제공하는 리소스 단위**로 공유할 수 있다. 비개발자도 변경할 수 있어 적용 절차가 비교적 단순하다.

리소스	공유 방식	사용 예
Prompt 리소스	ID 환경변수 주입 → 여러 Step-MCP-Code Serving 에서 참조	시스템 메시지, RAG 답변 템플릿
PyPI 패키지	MCP 서버 등록 시 체크박스 선택	사내 업무용 클라이언트, 공통 함수
JAR 패키지	Spring AI MCP 등록 시 선택	Spring Boot 공통 컴포넌트
Resource Group	리소스의 소속과 접근 권한을 관리	도메인별 리소스 소유·접근 범위
Serving (LLM 모델)	LLM_SERVING_ID와 실제 모델 식별자를 공유하여 같은 모델·과금·호출 횟수 제한 설정 사용	사내 vLLM 또는 등록된 외부 모델
MCP Server	server_id 공유 (9-3 패턴 B)	검색 도구 묶음, 사내 API 도구
A2A Workflow	agent_card_id를 환경변수로 전달하고 /a2a/<agent_card_id>로 호출	특정 도메인 에이전트

10.11 공통 모듈 적용 및 운영 기준

10.11.1 추출 제외 기준

- **입력값이 자주 바뀌는 코드** — 변경할 때마다 함수 인자가 달라지면 공통 패키지의 여러 사용처를 함께 수정해야 한다. 입력값이 정해진 후 분리한다.
- **1회용 임시 코드** — 회고, 분석 또는 일회성 실행에만 사용하는 코드는 공통 모듈로 분리하지 않는다.
- **분리했을 때 관리할 항목이 더 많아지는 코드** — 한 팀에서만 사용하는 코드는 같은 저장소에서 관리할 수 있다. 다른 팀에서도 사용해야 할 때 공통 모듈 분리를 검토한다.
- **GenOS 리소스로 참조할 수 있는 기능** — Prompt-PyPI-JAR-MCP Server-A2A Workflow 참조로 요구사항을 충족할 수 있으면 별도 패키지보다 기존 리소스 사용을 우선 검토한다.

10.11.2 추출 판단 기준

- 동일한 외부 API 인증, 입력 형식 또는 오류 변환 코드를 둘 이상의 서비스가 사용하고 함께 변경해야 하는 경우.

- 같은 dict key와 입력·출력 항목이 여러 업무에서 반복되는 경우.
- 동일한 오류 처리 및 재시도 패턴이 반복되는 경우.
- 새로 만든 MCP 도구가 기존 도구와 이름만 다르고 처리 내용이 비슷한 경우.

10.11.3 적용 및 운영 절차

1. 최초 구현은 해당 MCP, Workflow 또는 Code Serving 안에 작성한다.
2. 여러 사용처를 함께 변경해야 하면 담당 팀과 공개 입력·출력 형식을 정한다.
3. 공통 패키지로 분리하는 경우 PyPI, Verdaccio 또는 JAR 레지스트리에 게시하고 의존 사용처를 기록한다.
4. 호환성 변경과 제거 일정은 사내 패키지 운영 기준에 따라 공지하고 사용처 변경을 PR로 추적한다.

사용 중단 절차

- 공개 함수의 이름과 인자는 사용하는 코드를 변경하기 전에 삭제하지 않는다.
- README 또는 변경 기록에 사용을 중단할 대상, 대신 사용할 API 및 제거 예정일을 기록한다.
- 확인된 의존 사용처의 변경이 완료된 후 제거한다.

11. 배포 및 확인

11.1 배포 전 확인 사항

영역별 실행 함수, 패키지, 환경변수, 반환 형식 및 금지 사항을 확인한 후 배포한다. 세부 확인 항목은 부록 C를 따른다.

11.2 영역별 배포 방식

개발 영역	배포 방식
MCP 도구 (INTERNAL_PYTHON)	도구 > MCP 도구에서 코드와 패키지를 등록한 뒤 MCP 서버에서 해당 도구를 선택하여 배포
워크플로우 Python 단계	워크플로우 리비전에서 Python 단계 추가로 등록하고 배포
코드 서빙	코드 서빙과 리비전을 생성하고 Git 저장소·브랜치·커밋 해시 및 환경 설정을 등록하여 배포
MCP 패키지 (CONFIG_JSON)	npm 패키지를 게시하고 서버 타입 MCP 패키지로 등록
MCP Spring AI	fat JAR를 업로드하여 MCP 서버로 배포
Flowise Custom JS Function	워크플로우에 Utilities > Custom JS Function 노드를 추가하고 배포
전처리기	문서 전처리 > 전처리기 > 전처리기 생성에서 코드, 확장자 및 파라미터를 등록
평가지표	평가지표 생성에서 데이터셋 리비전과 코드를 등록하고 평가 실행 시 서빙 API 또는 워크플로우를 선택

11.3 실행 상태 확인

배포 후 기본 동작은 다음 항목으로 확인한다.

영역	최소 합격 조건
MCP	tools/list에 예상 Tool 이름·description·입력 항목이 표시되고 정상 입력의 tools/call이 성공
워크플로우	일반 함수는 문서에 정의한 필수 결과 key를 반환하고, generator는 마지막에 event: result를 반환
코드 서빙	/health가 HTTP 200을 반환하고 실제 업무 경로의 정상 입력·입력 오류·외부 시스템 timeout 결과가 API 문서와 일치
Flowise Custom JS Function	정상 입력의 반환값이 다음 Node에서 기대한 타입으로 수신되고 10초 안에 종료
전처리기	정상·빈 파일·손상 파일·파라미터 경계 테스트 결과가 예상과 일치하고 변경 파일이 재적재됨
평가지표	최대 3개 행의 평가지표 테스트가 통과하고 전체 평가의 오류 행 수와 점수 분포가 상세 설명과 일치

코드 서빙은 health 응답만으로 업무 기능 배포를 완료한 것으로 판단하지 않는다. 다음과 같이 정상, 입력 검증 실패 및 외부 연계 실패를 각각 실행한다.

```
GET /health → 200, {"status": "ok"}
POST /customers 정상 입력 → 200, customer_id 포함
POST /customers 필드 누락 → 422
CRM timeout → 504 또는 API 문서에 정의한 외부 시스템 timeout 응답
```

11.4 로그 확인

영역별 표준 출력 규칙에 따라 컨테이너 또는 실행 로그를 확인한다. 디버깅용 출력은 표준 로거를 사용하며 stdout 프로토콜을 사용하는 실행 영역에서는 stdout을 오염시키지 않는다.

11.5 주요 오류 및 조치 방법

11.5.1 MCP 도구가 tools/list에 표시되지 않는 경우

- 사용자 코드가 GenOS 제공 mcp 객체에 @mcp.tool() 함수를 등록하는지 확인한다. FastMCP를 다시 import·생성하지 않고, 도구 함수의 타입힌트와 decorator를 확인한다. Container log에서 도구 추출·등록 오류를 확인한다.

11.5.2 워크플로우 Python 단계에서 run function not found가 발생하는 경우

- 함수 이름이 정확히 run인지 확인한다. Run, main, execute는 진입점으로 인식되지 않는다.
- 들여쓰기 또는 구문 오류로 함수가 정의되지 않은 경우에도 동일 메시지가 발생할 수 있으므로 Python 구문을 확인한다.

11.5.3 코드 서빙 배포에서 health check가 실패하는 경우

- 사용자 웹 애플리케이션에 GET /health가 정의돼 있고 HTTP 200을 반환하는지 로컬에서 먼저 확인한다.
- 서버가 0.0.0.0:\$PORT에 bind하는지, 시작 (Run) 커맨드가 foreground HTTP process를 실행하는지, 빌드 산출물 경로가 명령과 일치하는지 확인한다.

11.5.4 Secret 관리 위치

- MCP 도구는 중요 정보 > 환경 변수, 워크플로우 Python 단계는 리비전 정보 > 환경 변수, 코드 서빙은 코드 서빙 리비전 상세 > 환경 설정 > 환경 변수, 전처리기는 생성·수정 화면의 환경 변수, Flowise Custom JS Function은 워크플로우의 \$vars를 사용한다. JS/TS-Spring AI MCP 패키지의 서버 설정 > env에는 비밀이 아닌 실행 설정만 입력하고, 시크릿은 운영 환경에서 정한 보호된 환경변수 주입 방법을 사용한다. 코드·저장소·패키지·JAR에는 평문 자격증명을 기록하지 않는다.

11.5.5 Streaming 응답이 일괄 반환되는 경우

- 워크플로우 Python 단계에서 단일 return 함수는 중간 token을 전달하지 않는다. streaming이 필요하다면 sync 또는 async generator의 run(data)에서 yield {"event": "token", "data": "...}를 보내고 마지막에 yield {"event": "result", "data": "...}를 보낸다.

- 코드 서빙은 사용하는 프레임워크의 실시간 응답 기능을 사용하고 호출 프로그램이 중간 결과를 받는지 확인한다. 일반 JSON 응답은 처리가 끝난 후 한 번에 전달된다.

11.5.6 외부 npm/PyPI package 추가 방법

- MCP에서는 관리 > 리소스 > PyPI 패키지에 .whl 파일을 업로드한 후 MCP 도구에서 선택한다.
- 워크플로우 Python 단계에서는 워크플로우 pod의 기본 이미지에 포함된 패키지만 사용할 수 있다. 다른 패키지가 필요하면 운영 팀에 기본 이미지 갱신을 요청한다.
- 코드 서빙에서는 언어별 의존성 파일과 lock file을 갱신하고 빌드 (Build) 커맨드에서 동일 설치 명령을 실행한다. 폐쇄망이면 해당 패키지 registry 또는 mirror 접근 여부를 확인한다.

11.5.7 Log가 표시되지 않는 경우

- `print()` 대신 `from common.logger import Logger; logger = Logger.getLogger(__name__); logger.info(...)`를 사용한다. 출력된 로그는 컨테이너 stdout을 통해 GenOS 로그 시스템에 수집된다.
- Trace 연계: Workflow가 주입한 `data["genos_state"] ["trace_id"]`를 애플리케이션 로그에 포함하면 요청 간 로그를 같은 식별자로 검색·연계할 수 있다. GenOS Trace 화면은 `genos_trace_id`로 상세 URL을 생성하지만, 임의의 로그 extra 필드가 자동으로 클릭 가능한 링크가 된다고 가정하지 않는다.

11.5.8 다른 GenOS Service 호출 방법

- Workflow에서 MCP, Code Serving 또는 다른 Workflow를 호출할 때는 **Gateway 경로**(`/api/gateway/...`)와 **Bearer 토큰**을 사용한다. 인증과 실행 기록을 거치지 않는 K8s service DNS 직접 호출을 금지한다.

11.5.9 JS/TS MCP의 tools/call 응답이 지연되는 경우

- CONFIG_JSON은 supergateway stdio bridge를 사용한다. 사용자 package에서 `console.log`로 stdout에 일반 log를 출력하면 JSON-RPC frame이 손상될 수 있으므로 debug log는 `console.error(stderr)`를 사용한다.

11.5.10 Spring AI MCP Pod의 readiness 실패가 반복되는 경우

- `spring-boot-starter-actuator`가 누락되었거나 `/actuator/health`가 401 또는 403 응답을 반환하는지 확인한다. `management.endpoints.web.exposure.include=health` 설정으로 상태 확인 경로가 공개되어 있는지도 확인한다.
- Java 실행 환경은 Spring Boot HTTP 서버를 직접 실행한다. `SERVER_PORT`를 application 설정에서 사용하고 `java -jar`로 실행 가능한 JAR인지 확인한다.

11.5.11 Flowise Custom JS Function에서 `require('axios')`가 실패하는 경우

- Flowise 컨테이너에서는 환경변수 `T00L_FUNCTION_EXTERNAL_DEP`의 허용 목록에 포함된 패키지만 `require`로 불러올 수 있다. 필요한 패키지가 목록에 없으면 운영팀에 추가를 요청한다.
- Node 내장 모듈(`crypto`, `url` 등)은 `T00L_FUNCTION_BUILTIN_DEP`에 등록돼야 한다.

11.5.12 JS/TS MCP package 실행이 실패하는 경우

- `scoped·unscoped` 여부와 관계없이 `package.json`의 `bin`이 실제 게시된 실행 파일을 가리키는 지 확인한다.
- `npm pack --dry-run`으로 `dist`와 실행 파일이 package에 포함되는 지 확인하고, GenOS와 같은 `npx -y <package>` 명령을 로컬에서 실행한다.

부록 A. 영역별 실행 함수 요약

개발 영역	사용자 작성 단위	필수 이름·등록 방식	반환
MCP 도구 (INTERNAL_PYTHON)	Python 함수	시스템 제공 mcp 객체의 <code>@mcp.tool()</code>	JSON으로 바꿀 수 있는 값
워크플로우 Python 단계	Python 함수 또는 generator	<code>run(data)</code>	dict 또는 스트리밍 이벤트
코드 서빙	Git 저장소 기반 웹 애플리케이션	시작 (Run) 커맨드, <code>0.0.0.0:\$PORT</code> , health endpoint	HTTP response
MCP 패키지 (CONFIG_JSON)	JavaScript/TypeScript handler 또는 Python stdio MCP 서버	<code>server.tool(...)</code> , Python 모듈 또는 스크립트 코드	MCP content 응답
MCP Spring AI	Java 메서드	<code>@Tool</code> , <code>@ToolParam</code>	JSON으로 바꿀 수 있는 단순 값 또는 응답 DTO
Flowise Custom JS Function	JavaScript 코드 블록	명시적 <code>return</code>	객체·문자열·숫자
전처리기	Python 클래스	<code>DocumentProcessor.__call__</code>	<code>text</code> 를 포함한 <code>list[dict]</code>
평가지표	Python 함수	<code>async def evaluate(row, predict)</code>	숫자 점수

A.1 GenOS 관리형 Python 코드 영역 비교

항목	MCP 도구 (INTERNAL_PYTHON)	워크플로우 Python 단계
실행 함수	시스템 제공 mcp 객체에 <code>@mcp.tool()</code> 함수 다수 등록	<code>def</code> 또는 <code>async def</code> <code>run(data)</code> 1개, 필요 시 generator
함수명 강제	외부 tool id는 <code>{domain}_{action}_{resource}[_{qualifier}]</code>	<code>run</code> 고정
함수 인자	Tool별 업무 입력값	<code>data: dict</code> 1개
반환	JSON으로 바꿀 수 있는 값	dict 또는 <code>yield event/data</code>
async 적용	외부 I/O 도구에 권장	sync, coroutine, sync/async generator 지원
타입 힌트	입력 형식을 만들기 위해 작성	권장
Docstring	Tool 설명과 입력값 작성	입력값과 반환값 작성
시크릿	<code>os.environ[KEY]</code>	<code>os.environ[KEY]</code>
패키지 추가	관리 > 리소스 > PyPI 패키지 사전 등록	workflow image에 포함된 package 사용
로깅	<code>Logger.getLogger(__name__)</code>	<code>Logger.getLogger(__name__)</code>
timeout	외부 호출에 명시	외부 호출에 명시

A.2 다중 언어 영역 비교

항목	JS/TS MCP 패키지 (CONFIG_JSOP)	Spring AI MCP (Java)	Flowise Custom JS Function (JS)
실행 함수	server.tool(name, desc, zod, async handler) 다수	@Tool + @ToolParam 어노테이션 메서드 다수	IIFE 본문 — return value
함수명 강제	tool 이름 {domain}_{action}_{resource} [_{qualifier}]	Java 메서드는 lowerCamelCase, @Tool(name=)의 외부 이름은 공통 형식	함수 자체 정의 불필요
반환	{ content: [{ type: "text", text: "..."}] }	JSON으로 바꿀 수 있는 단순 값 또는 응답 DTO	string / number / object / array
async 적용	필수 (Promise handler)	동기·비동기 지원	자동 적용 (async IIFE)
입력 확인	zod로 입력 항목 정의	@ToolParam(description="...", required=...)	typeof 등을 사용하여 직접 확인
시크릿	보호된 방법으로 주입 후 process.env.KEY로 조회	보호된 방법으로 주입 후 @Value("\${KEY}")로 조회	\$vars.KEY (UI 변수)
패키지 빌드	npm publish → Verdaccio	mvn package fat JAR → NFS 업로드	(없음 — 코드 인라인)
로깅	console.error (stderr 만)	LoggerFactory.getLogger	console.log / console.error
오류 처리	isError: true와 3.9절의 JSON 오류 사용	catch 적용 후 3.9절의 JSON 오류 반환	return { error: { error_code, msg, retryable } } 사용
GenOS DB 저장 여부	저장하지 않음 — package가 원본	저장하지 않음 — JAR가 원본	Workflow Step에 인라인 저장
컨테이너	supergateway 안 stdio process	Spring Boot HTTP process 직접 실행	Flowise 컨테이너 vm2 NodeVM
실행 명령어	command: npx	command: java	(해당 없음)

A.3 Python 데이터 처리 영역 비교

항목	전처리기	평가지표
실행 함수	DocumentProcessor.__call__(request, file_path, **kwargs)	evaluate(row, predict)
생성·호출 방식	DocumentProcessor() 생성 후 비동기 호출	데이터셋 행마다 비동기 호출
입력	업로드 파일 경로, Request, 전처리 파라미터	데이터셋 행, LLM 또는 Workflow 예측 함수
반환	text를 포함한 list[dict]	float()로 변환 가능한 숫자
주요 오류	지원하지 않는 형식, 빈 문서, parser 실패	필수 column 누락, 빈 기준값, 예측 호출 실패
변경 후 확인	대표 파일 실행 및 AI Drive 재적재	최대 3개 행 평가지표 테스트 후 전체 평가

A.4 코드 서빙 실행 영역

항목	실행 기준
지원 언어	Python (FastAPI / Flask / Django), Java (Spring Boot), JavaScript (Node / Express / Nest / Fastify), Go (net/http / Gin / Echo / Fiber), C# (ASP.NET Core)
배포 단위	Git 저장소와 고정 commit hash
빌드	환경 설정 > 빌드 (Build) 커맨드
기동	환경 설정 > 시작 (Run) 커맨드 또는 Python main.py/src/main.py 자동 탐색
네트워크 설정	0.0.0.0:\$PORT 연결, /health 200 응답
업무 API	사용자 애플리케이션이 HTTP 경로와 요청·응답 항목 정의
플랫폼 연계 path	필요 시 /chat, /preprocess, /mcp/list, /mcp/call 구현
Python 호환 방식	service(config, data)와 /json·/multipart는 기존 코드 지원용이며 공통 필수 형식이 아님

부록 B. 시스템 제공 영역과 사용자 작성 영역

영역	시스템이 자동으로 해주는 것	엔지니어가 작성하는 것
MCP 도구 (INTERNAL_PYTHON)	FastMCP 인스턴스, /mcp HTTP 연결, 실행 시작·종료 처리, /healthcheck, K8s Pod, PyPI 설치, 환경 변수 주입, tools/list 입력 형식 생성	@mcp.tool() 함수 본체, docstring, 타입힌트
워크플로우 Python 단계	data 입출력, genos_state 주입 (trace_id/session_id/user_id), OpenTelemetry tracer/span, exec 스코프, generator 분기, 오류 자동 로깅	run(data) 함수 본체, 비즈니스 로직, yield 이벤트 구조
코드 서빙	언어별 기본 이미지, Git 저장소 가져오기, 빌드·실행 커맨드 수행, 환경 변수 주입, Gateway 연결	사용자 웹 애플리케이션, 업무 경로와 요청·응답 항목, /health, 의존성·lock file, 필요 시 GenOS 연계 HTTP 경로
MCP 패키지 (CONFIG_JSON)	supergateway stdio↔HTTP 브릿지, K8s Pod, NPX-Python 실행 명령 수행, env 주입, tools/list 자동 동기화	npm 패키지의 server.tool() 등록과 bin 정의 또는 Python 모듈·스크립트 코드로 구현한 stdio MCP 서버
Spring AI MCP	JAR NFS 복사, Java 21 실행 환경, SERVER_PORT 주입, Spring Boot HTTP 프로그램 실행, K8s Pod, Tool 입력 형식 생성	Spring Boot 애플리케이션, @Tool + @ToolParam 메서드, MethodToolCallbackProvider Bean, 상태 확인 기능, 실행 가능한 JAR
Flowise Custom JS Function	vm2 NodeVM 샌드박스, \$input/\$vars/\$flow 주입, async IIFE 자동 래핑, 이전 노드 출력 연결	JS 코드 본문, return 값, 외부 호출 timeout, DB 커넥션 close
전처리기	업로드 파일 경로와 화면의 파라미터 전달, 반환 chunk의 임베딩 연계, 전처리기 본문 변경 시 재적재 대상 표시	DocumentProcessor 클래스, 지원 확장자별 parser, chunk와 부가 정보 구성, 임시 파일 정리
평가지표	데이터셋 행 순회, LLM 또는 워크플로우 predict 함수 전달, 점수 변환 및 오류 행 집계	evaluate(row, predict) 함수, 필수 column 처리, 평가지표별 점수 계산

부록 C. 영역별 개발 확인 항목

C.1 MCP 도구 (INTERNAL_PYTHON)

- [] 사용자 코드에서 FastMCP를 새로 만들지 않고 시스템 제공 mcp 객체에 @mcp.tool() 등록
- [] 외부 I/O 도구는 async def, sync 도구에는 blocking I/O 없음
- [] 모든 파라미터에 타입힌트 + 반환 타입힌트
- [] docstring에 대상·조건·결과와 비슷한 Tool과의 선택 기준을 작성하고 Args:와 Returns:를 포함
- [] 외부 호출에 timeout
- [] 오류 응답과 내부 로그에 같은 error_code 사용, 예외 원문은 Tool 결과에서 제외
- [] 시크릿은 MCP 도구의 중요 정보 > 환경 변수에서 주입
- [] PyPI 패키지는 관리 > 리소스 > PyPI 패키지에 사전 등록
- [] tools/list 로 도구 추출 확인
- [] MCP 서버 테스트에서 1회 이상 호출 확인

C.2 워크플로우 Python 단계

- [] 함수 이름이 정확히 run

- [] 함수 인자가 `data: dict` 형식인지 확인
- [] `return` 값이 `dict` (또는 `generator` 의 마지막 `event: result` 의 `data` 가 `dict`)
- [] `data` 의 기존 키 보존 (`return {**data, ...}` 또는 직접 수정)
- [] 외부 I/O 는 `async + timeout`
- [] 스트리밍이면 ③·④ 패턴, 비스트리밍이면 ①·② 패턴 — 혼용 금지
- [] `input-required` 상태는 3채널 중 하나로 정확히 전달
- [] 오류는 예외 발생 또는 `data["error"]` 전달 중 처리 흐름에 맞는 방식을 선택
- [] `data["error"]`에는 `error_code`, `msg`, `retryable`을 사용하고 내부 예외 원문은 포함하지 않음

C.3 코드 서빙

- [] 언어 선택값이 `Python`, `Java`, `JavaScript`, `Go`, `C#`의 화면 선택값 중 하나인지 확인
- [] Git 저장소 URL과 재현 가능한 `commit hash` 등록
- [] 의존성·lock file과 빌드 (Build) 커맨드 일치
- [] 시작 (Run) 커맨드가 `foreground HTTP process` 실행
- [] `0.0.0.0:$PORT`에 bind
- [] `/health`가 `HTTP 200` 반환
- [] 환경 변수·시크릿은 코드 서빙 리비전 상세 > 환경 설정 > 환경 변수에서 주입하고 코드에는 직접 입력하지 않음
- [] 실제 업무 route의 정상·validation 실패·upstream timeout 응답 확인
- [] 오류 코드는 코드 서빙 영역 코드 03과 3.9절의 공통 오류 코드를 조합하고 내부 로그와 API 응답에서 같은 코드를 사용
- [] 내부 로그의 `error_type`은 `{DOMAIN}_{CATEGORY}_{REASON}` 형식으로 작성하고 사용자 응답에는 포함하지 않음
- [] 컨테이너 서비스에서 워크플로우·전처리기·MCP 서버 연계를 설정한 경우 등록 경로와 `/chat/preprocess/mcp/list/mcp/call` route 일치
- [] 전처리기로 연결한 경우 `file_content_base64`를 사용하여 파일을 읽고 `file_name·file_size`를 확인
- [] MCP 서버로 연결한 경우 목록과 호출 응답을 각각 `{"code":0,"data":{"tools":[...]}}`, `{"code":0,"data":{"content":[...]}}` 형식으로 반환
- [] 에이전트 구현 시 `router-service-Tool` 등록부와 공통 모듈을 분리
- [] Tool 호출 반복 횟수, 외부 호출 timeout 및 등록되지 않은 Tool 차단 확인
- [] Skill 디렉터리와 `name`은 동일한 `kebab-case`로 작성하고 상세 파일명은 정확히 `SKILL.md` 사용
- [] `description`에 처리 기능, 사용 조건, 사용자 요청 예와 사용하지 않는 조건 작성
- [] Skill 본문에 Tool 호출 순서, 분기, 예외 처리, 출력 형식 및 입력·출력 예 작성
- [] `name`과 `description`만 항상 제공하고 본문과 번들 리소스는 필요한 시점에만 제공

C.4 JavaScript/TypeScript 및 Python MCP 패키지

- [] 실행 템플릿을 `NPX` 패키지, `Python` 모듈 또는 직접 입력 중 구현 방식에 맞게 선택
- [] `Python` 모듈은 `python3 -m <module_name>`으로 실행하고, 단일 파일은 스크립트 코드와 스크립트 파일명을 일치시킴
- [] `@modelcontextprotocol/sdk` 사용 + `StdioServerTransport`
- [] 모든 Tool을 `server.tool(name, desc, zod_schema, async handler)`로 등록
- [] 도구 이름은 `{domain}_{action}_{resource}_{qualifier}` 형식으로 작성
- [] `description`에 대상·조건·결과, 비슷한 Tool과의 선택 기준 및 데이터 변경 여부 포함
- [] 각 입력 항목의 `description`에 형식·범위·단위·기본값 포함
- [] 반환은 `{ content: [{ type: "text", text: "..."} ] }`
- [] `package.json`의 `"bin"`이 게시된 실행 파일을 가리키고 `"files"`에 빌드 산출물이 포함됨
- [] ESM을 사용하면 `"type": "module"` 지정, CommonJS와 혼용하지 않음
- [] `scoped` 여부와 관계없이 `package.json`의 `bin`이 게시된 실행 파일을 가리킴

- [] 서버 설정 > env에는 비밀이 아닌 실행 설정만 입력하고, 시크릿은 운영 환경에서 정한 보호된 환경변수 주입 방법으로 전달
 - [] console.log 금지, console.error 만 사용
 - [] 외부 호출에 timeout (AbortSignal.timeout)
 - [] 오류 응답과 stderr 로그에 같은 error_code 사용, 예외 원문은 Tool 결과에서 제외
-

C.5 Spring AI MCP

- [] Spring AI 1.1.3+ / Spring Boot 3.x / Java 21 실행 환경과 호환
 - [] spring-ai-starter-mcp-server-webmvc + spring-boot-starter-actuator
 - [] @Tool에 공통 외부 이름, 대상·조건·결과와 Tool 선택 기준을 작성하고 @ToolParam에 형식·범위·단위·기본값 포함
 - [] 메서드 반환은 JSON으로 바꿀 수 있는 단순 값 또는 필드가 정의된 응답 DTO
 - [] @Service 클래스가 MethodToolCallbackProvider 에 등록
 - [] java -jar로 실행 가능한 fat JAR이며 JAR 패키지 선택에서 지정한 파일과 업로드 파일이 일치
 - [] application.yml 시크릿은 \${ENV_VAR} 만 사용
 - [] HTTP 호출 timeout 설정
 - [] 광범위 catch (Exception) 금지 — 구체 예외 + 로그
 - [] 오류 응답과 내부 로그에 같은 error_code 사용, 예외 원문은 Tool 결과에서 제외
-

C.6 Flowise Custom JS Function

- [] 시크릿-URL은 \$vars로 받고 GenOS 호출에는 \${GENOS_URL}/api/gateway/... 경로 사용
 - [] 외부 호출에 timeout 명시
 - [] DB 커넥션은 finally { conn.end() }
 - [] JSON.parse 전에 typeof 체크
 - [] 요청 간 상태를 globalThis에 저장하지 않고 외부 저장소 사용
 - [] 파일 입력을 처리할 때 앞선 Node와 정한 type·data 형식 확인
 - [] 오류 반환 시 { error_code, msg, retryable }을 사용하고 예외 원문은 제외
-

C.7 전처리기

- [] 클래스명을 DocumentProcessor로 지정하고 DocumentProcessor()로 인자 없이 생성 가능
 - [] async def __call__(request, file_path, **kwargs)를 정의하고, 모든 chunk에 비어 있지 않은 문자열 text가 있는 list[dict] 반환
 - [] 외부 API 설정과 시크릿은 전처리기 화면의 환경 변수에 등록하고 코드에서는 os.getenv() 또는 os.environ으로 조회
 - [] 등록 확장자와 실제 parser 지원 확장자 일치
 - [] 원본 파일을 덮어쓰지 않고 임시 파일을 finally 또는 TemporaryDirectory로 정리
 - [] 화면의 파라미터와 kwargs key를 일치시키고, 코드·확장자·파라미터 변경 후 AI Drive 재적재 실행
-

C.8 평가지표

- [] async def evaluate(row, predict) 정의
 - [] 데이터셋 리비전의 실제 column에서 입력값을 읽고, 행마다 동일 입력에 대해 await predict(question)을 한 번만 호출
 - [] 상세 설명에 적은 범위와 방향에 맞는 숫자 점수를 반환하고 None 또는 오류 문자열은 반환하지 않음
 - [] 필수 column 누락과 빈 기준값을 구체적인 예외로 처리하고, 최대 3개 행의 평가지표 테스트 후 전체 평가 오류 행 수 확인
-